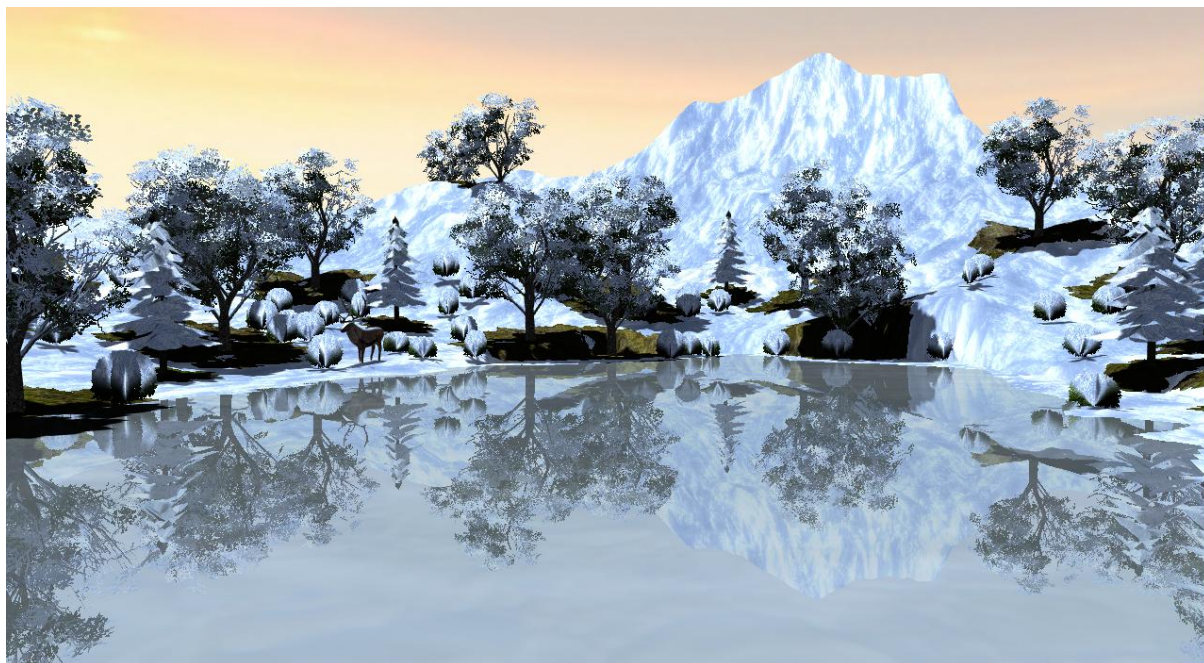


WINTER

Γραφικά και Εικονική Πραγματικότητα



Ζωγράφου Μαρία Νίκη
1096060

Contents

PART A	3
Terrain	3
Clouds	6
Sky-dome Ουρανός	8
Lake.....	10
Directional light source, shadows	12
4. VEGETATION	14
Μοντέλα και υφές Δέντρων	14
Μοντέλο και υφές θάμνων	16
Mini maps για ποιότητα υφών	17
Forest creation	18
Instancing	18
Tree Density Mask	18
Εύρεση Σωστού Ύψους (Heightmap Integration).....	18
Έλεγχος Ελεύθερων Θέσεων και Συγκρούσεων.....	18
Bush creation.....	20
PART B	21
Snowfall	21
Fog	22
Snow accumulation	23
Snowflakes.....	23
Snow Texture.....	24
Snow Inflation.....	24
Frozen lake + Reflection	25
6. Wind	27
7. a. Clouds clear away	27
b. ice crack lake melts.....	28
c. snow melts.....	29
d. avalanche.....	30
Miscellaneous.....	32
Controls παιχνιδιού	32
Flying vs Walking mode F key	32
Interactions με ζώα	34
Full screen mode	35
Python Helpers	36

Βιβλιογραφία	36
--------------------	----

GitHub: <https://github.com/Manyaz1/Winter-with-OpenGL>

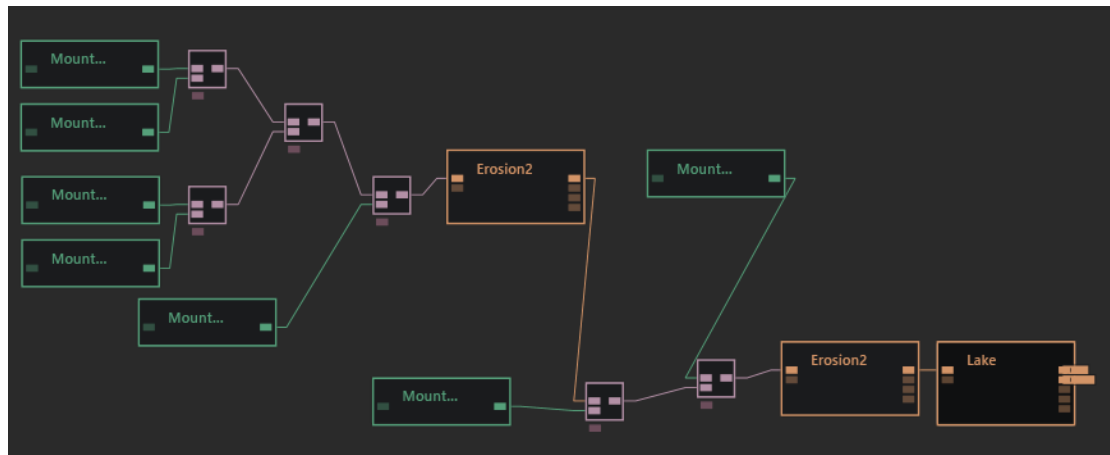
Video: <https://youtu.be/145MWfZbKs4> (To recording με το OBS χαμήλωσε τα fps)

Terrain

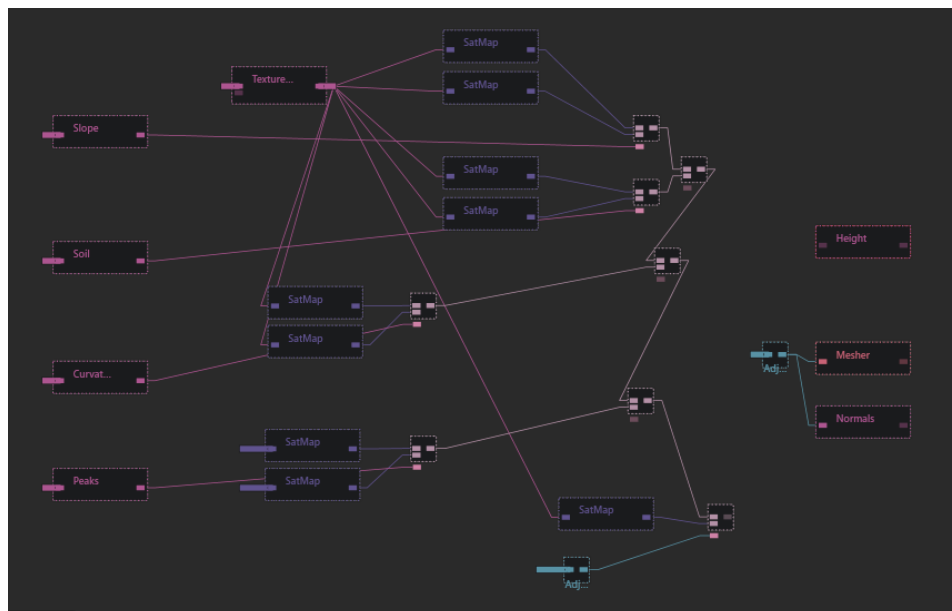
“Load a mountainous terrain (or generate one, if you prefer). ”

Το Terrain δημιουργήθηκε με το gaea και ύστερα φορτώθηκε από την OpenGL ως .obj αρχείο. Το gaea community edition είναι ένα εργαλείο για την παραγωγή εδαφών με πολλαπλές δυνατότητες: δημιουργία βουνών, λιμνών, ποταμών, προσομοίωσης φθοράς από χρόνο (erosion) και βάθος λιμνών από βροχόπτωσή καθώς και άλλες δυνατότητες.

Το terrain του project περιέχει 7 βουνά και μία λίμνη:



Επιπλέον παράχθηκαν κάποια αρχικά textures, ένα erosion map/ height map και η μάσκα της λίμνης.



Υπήρχε επιπλέον η δυνατότητα παραγωγής αντικειμένων obj διαφορετικής ακρίβειας (LOD1,2,3) τα οποία είχαν μικρότερη ακρίβεια αλλά και μικρότερο μέγεθος, οπότε φορτώνονταν πιο γρήγορα από το πρόγραμμα.

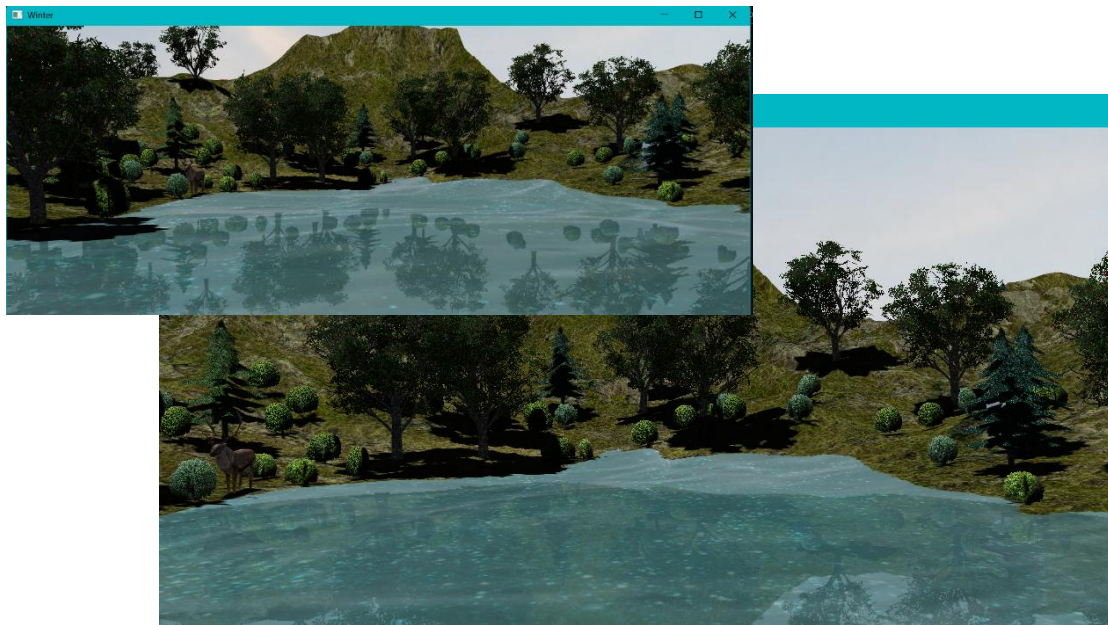
Πλεονέκτηματα: το Terrain είχε ήδη precomputed **normals** και **ρεαλιστική μορφή** (eroded mountains).

Μειονεκτήματα: Η λίμνη και το έδαφος ήταν κοινό αντικείμενο και υπήρχαν προβλήματα ακρίβειας στο ύψος της λίμνης και τον διαχωρισμό της με το υπόλοιπο λιβάδι.

Editing με Python:

Για την αντιμετώπιση των παραπάνω προβλημάτων χρησιμοποίησα κάποια βοηθητικά scripts για την επεξεργασία του terrain.

Κάποια σημεία της λίμνης, ακόμα και στο αντικείμενο της υψηλότερης ποιότητας, ανασηκώνονταν πάνω από το ύψος του νερού. Ύστερα από την προσθήκη αντανάκλασης στο Μέρος Β, αυτό το πρόβλημα έγινε πολύ αισθητό, καθώς τα παραμορφωμένα σημεία δεν λειτουργούσαν ως καθρέφτης.



Το [pythonFlatten.py](#), έχοντας το [lake_mask](#) από το gaea, επιτεδοποιεί το mesh στη μορφή της λίμνης που ορίζει η μάσκα, με ομαλές άκρες. Ουσιαστικά διορθώνει τις ατέλειες που προκαλούσαν το από πάνω πρόβλημα.

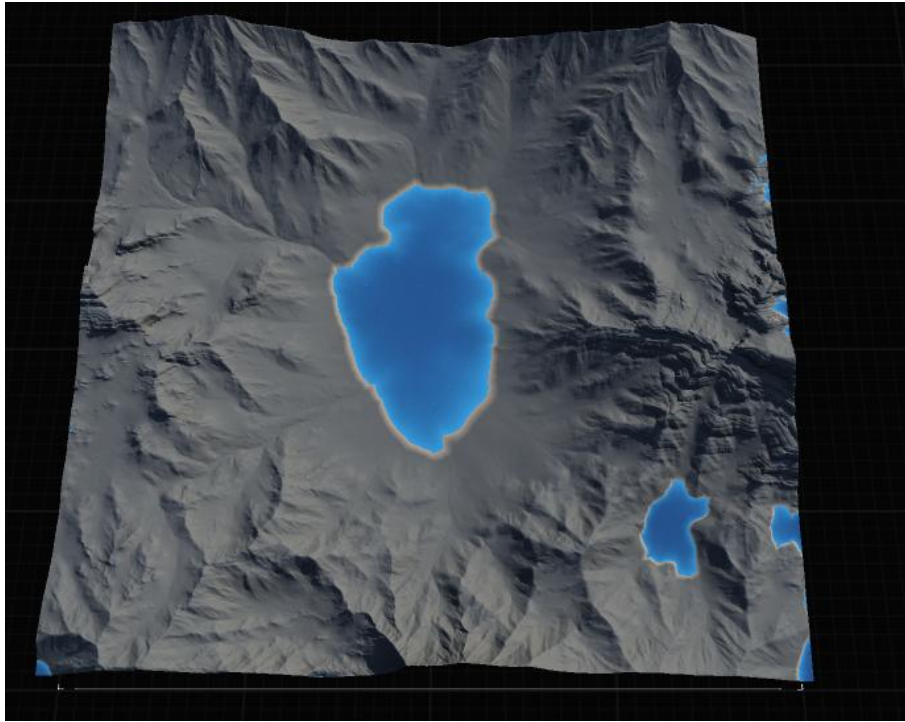


Το πρόγραμμα φορτώνει το 3D mesh (αρχείο OBJ) και την εικόνα μάσκας (BMP) σε grayscale. Η μάσκα ορίζει πού βρίσκεται η λίμνη: όπου τα pixel της μάσκας είναι πάνω από ένα όριο, θεωρείται περιοχή λίμνης. Για κάθε κορυφή (vertex) του mesh, εντοπίζει τη θέση της πάνω στη μάσκα. Αν η θέση αυτή ανήκει στη λίμνη (σύμφωνα με τη μάσκα), το ύψος της κορυφής (y) ορίζεται σε μια σταθερή τιμή (ύψος νερού), δηλαδή γίνεται επίπεδη. Αν η κορυφή είναι κοντά στο όριο της λίμνης,

το ύψος της προσαρμόζεται ομαλά μεταξύ του αρχικού ύψους και του ύψους του νερού, ώστε να υπάρχει ομαλή μετάβαση. Τέλος, αποθηκεύει το νέο mesh σε αρχείο και εμφανίζει πόσες κορυφές επηρεάστηκαν.

(δοκιμές με blur/χωρίς blur της μάσκας είχαν την ίδια απόδοση για τα lod2/lod3 αντικείμενα που χρησιμοποιήθηκαν τελικά).

Επιπλέον, επειδή το gaea δημιουργούσε μικρές λίμνες σε μεγάλο υψόμετρο, αφαιρέθηκαν από το lake mask με photo editing, παραμένοντας ως απλά σημεία του βουνού σαν κρατήρες στο τελικό project.



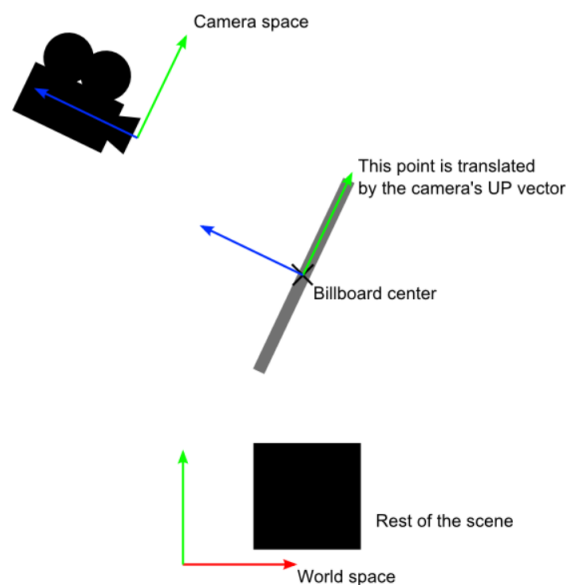
Clouds

“Add clouds in the sky and make them look as realistic as you can.”

Για τα σύννεφα είχα αναλογιστεί τρεις πιθανές κατευθύνσεις: 1 απλό texture στον ουρανό 2 εικόνες billboards με κίνηση και 3 volumetric clouds. Ως καλύτερη ισορροπία ανάμεσα σε ποιότητα και δυσκολία επέλεξα τα billboards.

“Billboards are 2D elements incrustated in a 3D world.” <https://www.opengl-tutorial.org/intermediate-tutorials/billboards-particles/billboards/>

The idea is that each corner of the billboard is at the center position, displaced by the camera's up and right vectors :



Στο αρχείο cloud.h, ορίζεται η δομή Cloud. Κάθε σύννεφο έχει τις εξής ιδιότητες:

- **position, size, alpha:** Θέση, μέγεθος και διαφάνεια.
- **speed:** Η ταχύτητα κίνησης.
- **textureID:** Επιλέγει ποιο από τα δύο διαθέσιμα textures (cloud.png ή cloud2.png) θα χρησιμοποιηθεί.

Έγινε χρήση δύο ημιδιαφανών εικόνων.

Για την φόρτωση Png με διαφάνεια χρειάστηκε να γίνει η εξής αλλαγή στην loadSOIL

```
GLuint loadSOIL(const char* imagePath) {  
    cout << "Reading image: " << imagePath << endl;  
    GLuint texture = 0;  
    //Load Image File Directly into an OpenGL Texture  
    texture = SOIL_load_OGL_texture  
    (  
        imagePath,  
        SOIL_LOAD_RGBA, //my change  
  
    TEXTURE = PNG APO SYNNEFO
```

εικόνες:



Ορίζουμε επίσης την κλάση `class CloudSystem`:

```
+ void initialize(GLuint shaderProgram);
```

Quads: Δημιουργεί ένα τετράγωνο (quad) χρησιμοποιώντας δύο τρίγωνα (quadVerts) και ορίζει τις UV συντεταγμένες (quadUVs).

Textures: Φορτώνει τις δύο εικόνες μέσω της loadSOIL και τις αποθηκεύει στον πίνακα cloudTextures[2].

Texture Parameters: Ρυθμίζει το GL_CLAMP_TO_EDGE για να μην εμφανίζονται γραμμές στις άκρες των σύννεφων και το GL_LINEAR για ομαλό φιλτράρισμα.

Uniforms: Συνδέεται με τον shader για να μπορεί να στέλνει το modelMatrix.

```
+ void addCloud(vec3 position, float size = 5.0f);
```

Νέο σύννεφο: Δημιουργία νέου σύννεφου με position, size, alpha, τυχαίο speed ανάμεσα στο 0.5 και 1.5 και textureID. Προσθήκη στην λίστα clouds.

```
+ void update(float deltaTime);
```

Ανανέωση θέσης: Ανανέωση της θέσης με βάση την ταχύτητα. Επίσης γίνεται έλεγχος να μην βγαίνουν εκτός των ορίων +-100 του terrain. Αν φτάσει στο +100 επιστρέφει στο -100.

```
+ void render(mat4 viewMatrix, mat4 projectionMatrix);
```

Billboarding: Για να φαίνονται τα σύννεφα τρισδιάστατα ενώ είναι 2D, το σύστημα υπολογίζει έναν πίνακα μετασχηματισμού που τα στρέφει πάντα προς την κάμερα:

Θέλουμε κάθε γωνία της πινακίδας να βρίσκεται στην κεντρική θέση, μετατοπισμένη από το πάνω και δεξί διάνυσμα της κάμερας.

Δηλαδή:

```
vec3 vertexPosition_worldspace=particleCenter_worldspace  
+CameraRight_worldspace*squareVertices.x*BillboardSize.x  
+CameraUp_worldspace*squareVertices.y*BillboardSize.y;
```

$$Vertex_{world} = x * col_0 + y * col_1 + z * col_2 + 1 * col_3$$

Φτιάχνουμε τις στήλες του πίνακα ως εξής:

```
billboard[0] = vec4(cameraRight * cloud.size, 0); // right vector * size  
billboard[1] = vec4(cameraUp * cloud.size, 0);  
billboard[2] = vec4(cameraForward * cloud.size, 0); // Z (βάθος).  
billboard[3] = vec4(cloud.position, 1);
```

Sky-dome Ουρανός

Αρχικά έθεσα το background χρώμα του καρέ σε γαλάζιο, όμως για πιο ρεαλιστική εμφάνιση το αντικατέστησα με skydome, δηλαδή μια σφαίρα που περικλείει τον κόσμο και έχει μια εικόνα στο εσωτερικό της.

Προκειμένου ο ουρανός να δίνει την αίσθηση του άπειρου και να μην μπορεί ο χρήστης να πλησιάσει στα τοιχώματά του, η μετατόπιση του skydome (model matrix translation) γίνεται πάντα με βάση τη θέση της κάμερας. Έτσι, η κάμερα παραμένει διαρκώς στο κέντρο της σφαίρας, κάνοντας τον θόλο να φαίνεται στατικός και απομακρυσμένος, ανεξάρτητα από την κίνηση στον χώρο.

Σχετικά με τη σειρά σχεδίασης, το skydome σχεδιάζεται πρώτο στη σκηνή. Παράλληλα, ρυθμίζεται κατάλληλα ο έλεγχος βάθους (depth function), ώστε όλα τα υπόλοιπα αντικείμενα (όπως το έδαφος ή η σφαίρα) να εμφανίζονται πάντα μπροστά από αυτόν.

Η υλοποίηση γίνεται στο `render_scene` του `winter.cpp` στο `// --- 1. SKY DOME --- section`.

BUG

Ένα πρόβλημα που αντιμετώπισα στο συγκεκριμένο ερώτημα ήταν το εξής:



Η αιτία ήταν πως στον αρχικό κώδικα του vertex shader γινόταν αυτό

Vertex shader

```
vertex_UV = spherical_mapping(vertexPosition);
```

Ο Vertex shader υπολόγιζε σωστό UV σε κάθε κορυφή και μετά η GPU έκανε γραμμική παρεμβολή (linear interpolation). Σε ένα σημείο του ορίζοντα η παρεμβολή ήταν λάθος λόγω κυκλικής γεωμετρίας.

$0.99 \rightarrow 0.01$ = μεγάλη διαδρομή προς τα ΠΙΣΩ

αντί για

$0.99 \rightarrow 1.00 \rightarrow 0.00 \rightarrow 0.01$ = μικρή διαδρομή ΓΥΡΩ ΑΠΟ ΤΗ ΣΦΑΙΡΑ

Επίλυση

Μεταφορά υπολογισμών στον fragmentshader ώστε η παρεμβολή να γίνεται **ΜΟΝΟ** στη **καρτεσιανή θέση (x,y,z)** και να υπολογίζει την πολική μετατροπή **ANA PIXEL**.

```
Fragment shader
void main() {
    if (useTexture == 3) {
        vec3 p = normalize(fragPos_modelspace);

        float pi = 3.14159265358979323846;

        // NOTE: use atan(z,x) so longitude runs correctly around sphere
        float u = atan(p.z, p.x) / (2.0 * pi) + 0.5;
        float v = asin(clamp(p.y, -1.0, 1.0)) / pi + 0.5;

        // fold into [0,1), safe for repeat-wrapped texture
        u = fract(u);
        v = clamp(v, 0.0, 1.0);

        fragmentColor = vec4(texture(skyTex, vec2(u, v)).rgb, 1.0);
        return;
    }
    else {
```

Lake

Έγινε χρήση κάποιων βασικών textures στην αρχή για τη λίμνη και το έδαφος. Ο διαχωρισμός ανάμεσα στα δύο πραγματοποιήθηκε με ένα lake mask

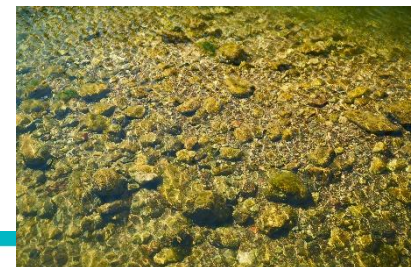
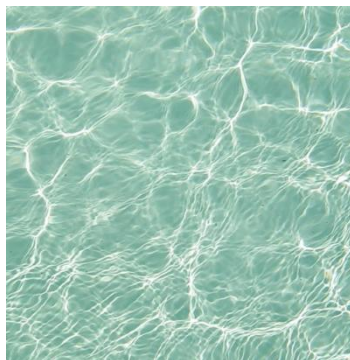
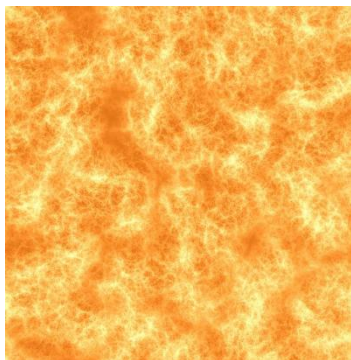
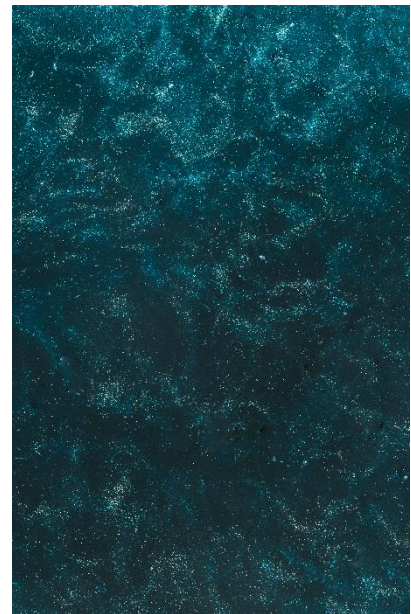
```
_Kd.rgb = mix(terrainColor, finalWater, lakeMask);
```

Σημείωση: Έχει χρησιμοποιηθεί μοντέλο Phong.

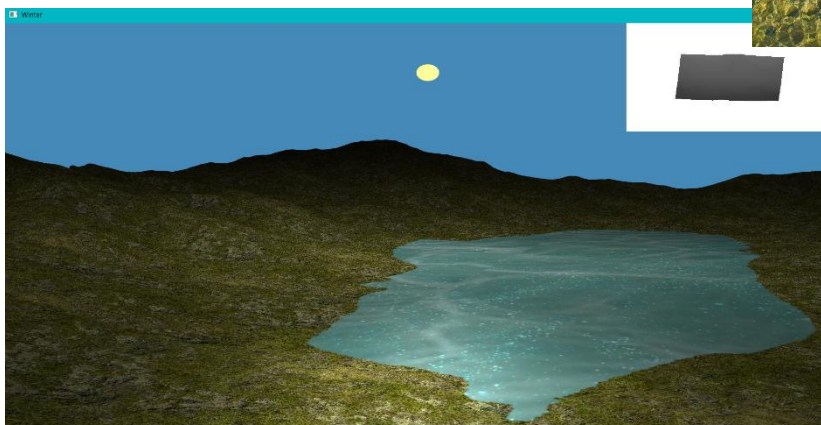
Για την εφαρμογή υφών σε άλλα αντικείμενα, τροποποιήθηκε η λογική του shader ως εξής:

```
uniform int useTexture = 0;
```

0. Απλά υλικά: (mtl struct)
1. Υφή terrain (λίμνη και έδαφος)
2. Άλλα αντικείμενα με υφή όπως Sun Sphere Texture



Αρχική Εμφάνιση:



Τα υν των δύο υφών της λίμνης μετακινούνται με sin και cos. Υπάρχει επίσης ένα κοινό waterflow συνημιτονοειδές κύμα που προστίθεται σε αυτά και αργότερα στην παραμόρφωση της αντανάκλασης.

```
vec2 waterFlow = vec2(time * 0.004, time * 0.005); //shared water flow
used for movement and distortion

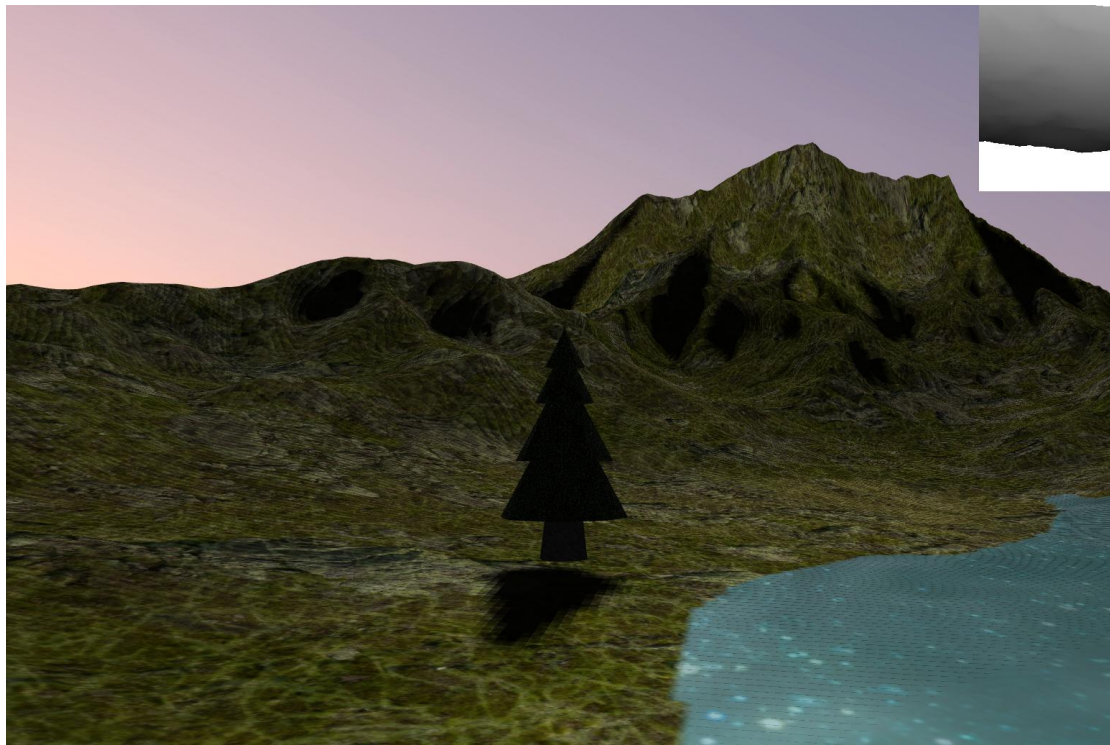
vec2 waterUV1 = vertex_UV + waterFlow +
vec2(
    sin(time * 0.3 + vertex_UV.y * 4.0),
    cos(time * 0.25 + vertex_UV.x * 3.0)
) * 0.003;

vec2 waterUV2 = vertex_UV - waterFlow * 0.8 +
vec2(
    cos(time * 0.35 + vertex_UV.x * 5.0),
    sin(time * 0.28 + vertex_UV.y * 4.0)
) * 0.002;

waterColor = mix(
    texture(waterTex, waterUV1).rgb, //lab texture
    texture(waterTex2, waterUV2* vec2(4.0,4.0)).rgb, //magical lake water
0.6);
```

Bonus: Reflection, Distortion παρακάτω

Directional light source, shadows



Έχει εφαρμοστεί το bidirectional lighting με μοντέλο Phong. Δηλαδή το φως είναι αρκετά μακριά ώστε οι ακτίνες του να είναι παράλληλες. Αυτό επιτεύχθηκε με αλλαγές στον κώδικα του εργαστηρίου.

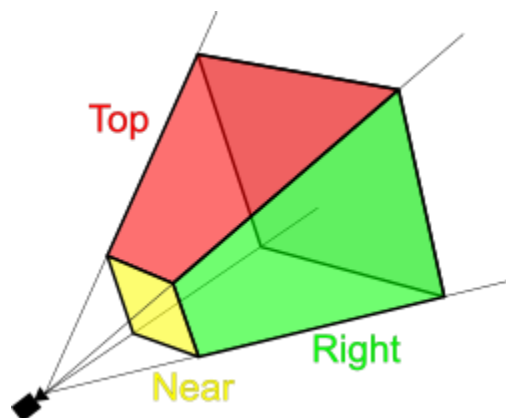
Αλλαγές στο Light.cpp:

```
// Δεν αποθηκεύουμε θέση, αλλά ΚΑΤΕΥΘΥΝΣΗ  
direction = normalize(init_direction); // Σταθερή κατεύθυνση  
// Για visualization (sun sphere στον ουρανό)  
sun_pos = -direction * radius; // Θέση sphere, ΟΧΙ του φωτός
```

Ισχύουν τα ίδια controls, τα οποία αλλάζουν την κατεύθυνση του φωτός και την θέση της σφαίρας ήλιου που χρησιμοποιείται για visualization. Επιπλέον γίνεται **ορθογραφική προβολή του φωτός** έτσι ώστε να έχουμε παράλληλες γραμμές προβολής (directional light).

Πρόβλημα:

Λόγω του μεγάλου μεγέθους του terrain, η ποιότητα των σκιών θύμιζε περισσότερο pixel art, παρά ρεαλιστικές σκιές. Αυτό γίνεται γιατί το shadow map έχει περιορισμένη ανάλυση. Λόγω του μεγέθους του terrain, κάθε pixel του shadow map αντιστοιχεί σε μια μεγάλη περιοχή του κόσμου.



Επίλυση: Directional light frustum fitting

Οτιδήποτε εκτός της κόλουρης πυραμίδα δεν χρειάζεται σκιές.

Οι σκιές υπολογίζονται από την οπτική γωνία του φωτός (**Light Space**). Επομένως, μετασχηματίζουμε τις γωνίες του frustum της κάμερας (view frustum corners) στο σύστημα συντεταγμένων του φωτός. Από αυτό βρίσκουμε τα **minX**, **maxX**, **minY**, **maxY**, **minZ**, **maxZ**. Αυτές οι τιμές μάς δίνουν το **μικρότερο δυνατό ορθογώνιο κουτί (orthographic box)** που περιέχει ό,τι μπορεί να δει η κάμερα του παίκτη, όπως αυτό φαίνεται από την οπτική γωνία του ήλιου.

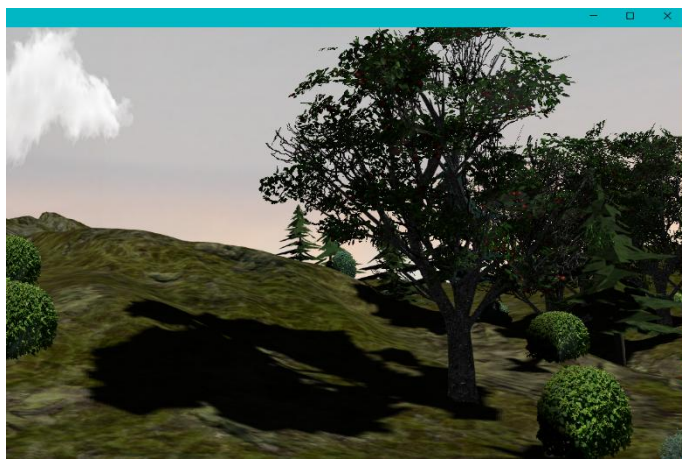
Έτσι περιορίζουμε το shadow map, αυξάνοντας την ποιότητα των σκιών.

Υλοποίηση στην `void Light::fitToCameraFrustum(const mat4& cameraView, const mat4& cameraProj)` του `light.cpp`

Πριν:



Μετά:



Σημείωση:

Το ίδιο το terrain έχει εξαιρεθεί από το depth pass, καθώς λόγω των προσεγγίσεων που έχουν γίνει, αν και λογικά ορθό το αποτέλεσμα δεν ήταν οπτικά «ρεαλιστικό/όμορφο» (οι σκιές από το βουνό είναι πιο σκοτεινές απ' ό,τι θα ήταν στην πραγματικότητα).



4. VEGETATION

Μοντέλα και υφές Δέντρων

1^η έκδοση:

Δημιούργησα τα παρακάτω δέντρα στο blender. Έγινε χρήση Instancing (εξήγηση παρακάτω) και 3 διαφορετικών υφών για τα φύλλα και μίας υφής για τον κορμό. Μετακίνησα τα UVs του κορμού στη αριστερή πλευρά του UV επιπέδου και τα υπόλοιπα στην δεξιά, έτσι ώστε να μπορώ να ελέγχω στον shader ποιο από τα δύο textures θα επιλεχθεί.

```
// Trunk or needles based on UV
if (vertex_UV.x < 0.5) {
    // TRUNK: Bark (same for all instances)
    vec2 barkUV = vec2(vertex_UV.x * 2.0, vertex_UV.y) * vec2(6.0, 6.0);
    _Kd.rgb = texture(trunkTex, barkUV).rgb;
} else {
```



BUG φωτισμού & επίλυση

ΧΡΕΙΑΣΤΗΚΕ ΑΥΤΟ: `vertex_normal_cameraspace = mat4(transpose(inverse(mat3(V * M)))) * vec4(vertexNormal_modelspace, 0);`

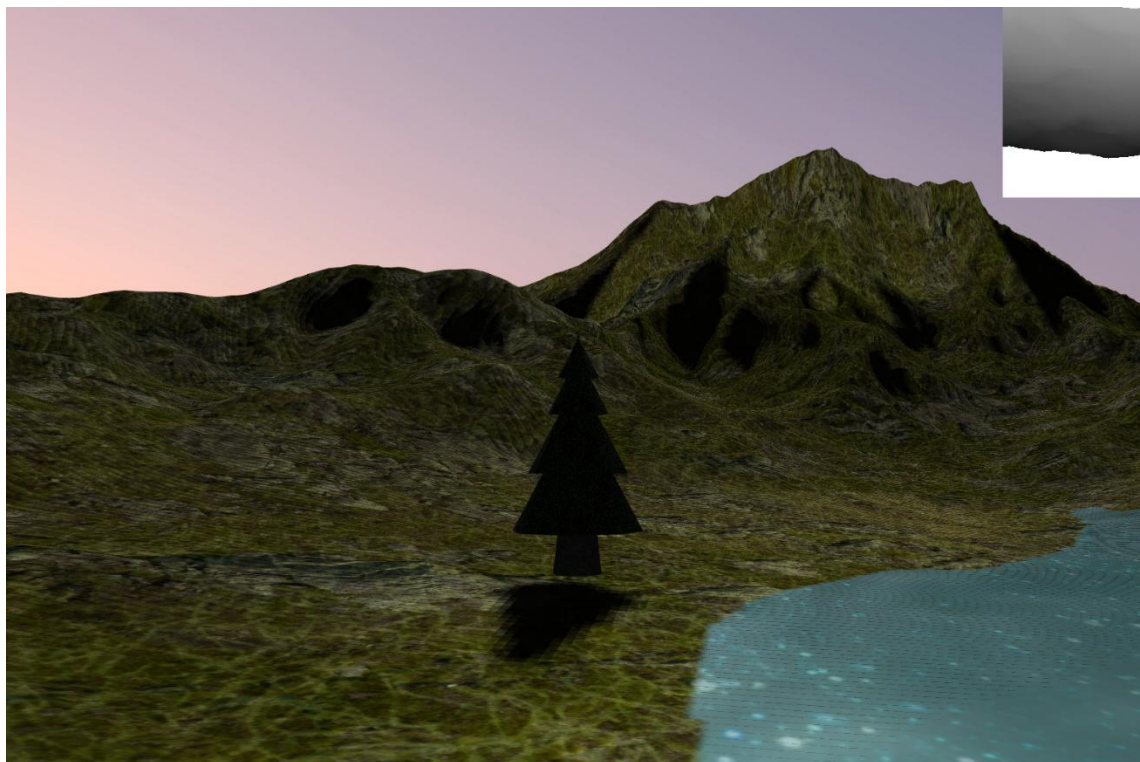
ΑΝΤΙ ΓΙΑ: `vertex_normal_cameraspace = V * M * vec4(vertexNormal_modelspace * normDir, 0);`

ΓΙΑΤΙ;

Τα Normals είναι σε camera space οπότε δεν είναι σωστό να πολλαπλασιάσουμε με $V*M$ (model-view). Πρέπει να πολλαπλασιαστούν με το **inverse-transpose** του **model-view**:

`vertex_normal_cameraspace = normalize(transpose(inverse(mat3(V*M))) * vertexNormal_modelspace);`

Αλλιώς το φως πάνω στους κώνους θα φαίνεται ανάποδα.



2^η έκδοση:

Επιλογή δύο ειδών δέντρων από cgtrader και sketchfab:

Μηλιά με ενιαίο texture και έλατα όπου κορμός και φύλλα ήταν χωριστά αντικείμενα. Με χρήση blender μετακίνησα πάλι τα UV του κορμού στην μία πλευρά και των φύλλων στην άλλη. Πάλι χρησιμοποιήθηκαν 3 διαφορετικές εικόνες για υφή φύλλων στα έλατα.

//5: Tree Texture

//6: Bush Texture

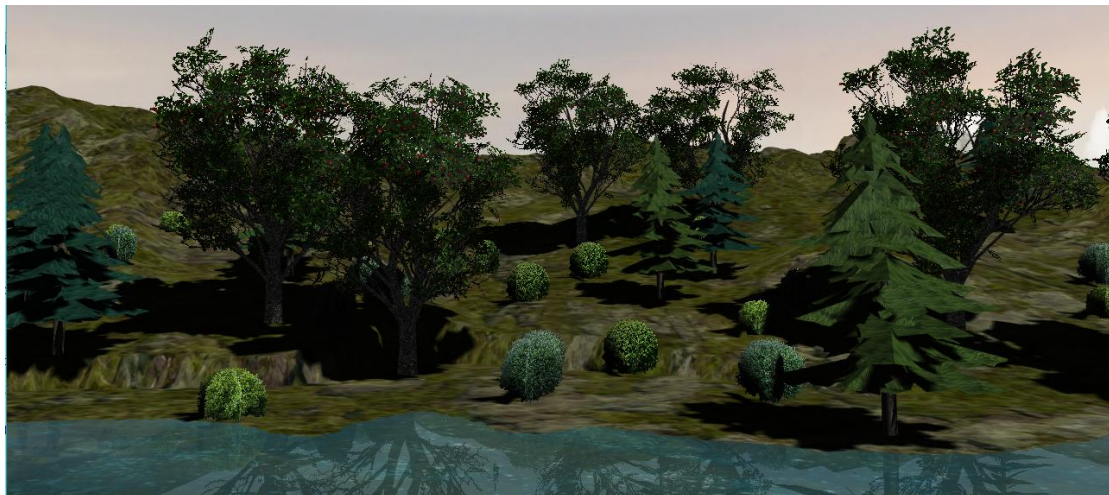
//7: Animal Texture (Deer, Bear) + apple [1 image texture -> 1 obj].

Στον fragmentShader για τη μηλιά θέτω την μεταβλητή `useTexture==7` καθώς θα κάνει χρήση ενιαίας υφής. Για τα έλατα θέτω 5 και κάνω διαχωρισμό ως εξής:

```

if (useTexture == 5) { // trees
// Trunk or needles based on UV
if (vertex_UV.x < 0.5) { // Επιλογή Ανάμεσα σε κορμό και φύλλα
// TRUNK: Bark (same for all instances)
vec2 barkUV = vec2(vertex_UV.x * 2.0, vertex_UV.y) * vec2(6.0, 6.0);
_Kd.rgb = texture(trunkTex, barkUV).rgb;
}
else {
// NEEDLES: Select texture based on instance index
vec2 branchUV = vec2((vertex_UV.x - 0.5) * 2.0, vertex_UV.y) *
vec2(9.0, 9.0); //repetition factor
vec4 texColor;
// Switch between different needle textures based on textureIndex
//Επιλογή διαφορετικών υφών με βάση uniform μεταβλητής textureIndex
if (textureIndex == 0) {
texColor = texture(needleTex, branchUV); // tree2.jpg
} else if (textureIndex == 1) {
texColor = texture(needleTex2, branchUV); // fir.jpg
} else {
//texColor = texture(needleTex2, branchUV);
texColor = texture(needleTex3, branchUV); // chrys.jpg
}
if (texColor.a < 0.1) discard;
_Kd.rgb = texColor.rgb;
}
}

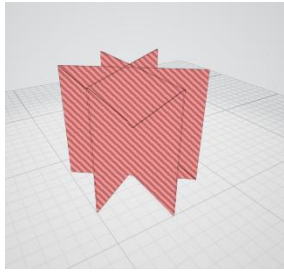
```



Μοντέλο και υφές θάμνων

Ως μοντέλο θάμνου χρησιμοποιήθηκαν 3 intersecting planes, πάνω στα οποία τοποθετήθηκαν 3 διαφορετικές εικόνες με διαφάνεια, έτσι ώστε από μακριά να λειτουργεί σαν billboard. Το πρόβλημα που προέκυψε ήταν τα νορμαλς στον φωτισμό, που δεν είχαν την κατεύθυνση που θα είχε ένας θάμνος. Για αυτό δημιούργησα μια αόρατη σφαίρα στο blender της οποίας τα normals χρησιμοποιεί το μοντέλο του θάμνου. Οπότε ο θάμνος έχει τα Normals και φωτισμό ημισφαιρίου.

Πάλι έχει γίνει χρήση 3 διαφορετικών εικόνων θάμνου για ποικιλία.



Mini maps για ποιότητα υφών

Έχουν δημιουργηθεί οι παρακάτω δύο συναρτήσεις στο winter.cpp

```
GLuint loadTextureRepeat_nominmap(const std::string& path)
```

Στέλνει στον shader REPEAT texture. ΔΕΝ παράγει minmap. Οι θάμνοι μετά την παραγωγή minmap είχαν έντομο μαύρο περίγραμμα, οπότε οι υφές των θάμνων φορτώνονται με αυτή τη συνάρτηση.

```
GLuint loadTextureRepeat(const std::string& path)
```

Στέλνει στον shader REPEAT texture και παράγει minmap. Δηλαδή όταν βλέπουμε μία υφή από μακριά θα χρησιμοποιείται μια έκδοση της με λιγότερη λεπτομέρεια.

each subsequent texture is twice as small compared to the previous one after a certain distance threshold from the viewer, OpenGL will use a different mipmap texture that best suits the distance to the object. Because the object is far away, the smaller resolution will not be noticeable to the user» από <https://learnopengl.com/Getting-started/Textures>

Αυτό έχει και “anti-aliasing” effect. Οι υφές των ελάτων επαναλαμβάνονταν πολλές φορές, επομένως όταν η κάμερα κουνιόταν μακριά από τα δέντρα, φαίνονταν σαν να «τρεμοπαίζουν». Όταν η κάμερα είναι κοντά στα δέντρα η συχνή επανάληψη (uv*6) είναι απαραίτητη για να είναι ρεαλιστικό, αλλά όταν βρισκόμαστε μακριά μια πιο απλή εικόνα με λιγότερη επανάληψη είναι προτιμότερη.

Χωρίς minmap:



Με minmap:



Forest creation

Με το αρχείο **forest.cpp** υλοποιείται η διαχείριση και η αποδοτική σχεδίαση μεγάλου αριθμού δέντρων (και θάμνων) στο περιβάλλον, χρησιμοποιώντας δεδομένα από το terrain για τη σωστή τοποθέτησή τους.

Instancing

Το δάσος δεν σχεδιάζει κάθε δέντρο ξεχωριστά αλλά χρησιμοποιεί τη τεχνική του Hardware Instancing μέσω της συνάρτησης `setUpInstancing()`. Δημιουργούνται Instance VBOs που αποθηκεύουν τους πίνακες μετασχηματισμού (`modelMatrices`) και τους δείκτες υφών (`textureIndices`), έτσι ώστε να έχουν τα δέντρα διαφορετικές θέσεις και υφές. Η σχεδίαση γίνεται με μία μόνο κλήση της `glDrawElementsInstanced`, μειώνοντας το overhead της CPU.

Δηλαδή το γεωμετρικό μοντέλο του δέντρου δίνεται μια φορά στη cpu για να σχεδιαστεί, μαζί με έναν πίνακα με 1.000 θέσεις. Όταν εκτελείται η `glDrawElementsInstanced`, η GPU τρέχει τον Vertex Shader για κάθε instance και σχεδιάζονται όλα ταυτόχρονα.

Tree Density Mask

Η τοποθέτηση των δέντρων γίνεται δυναμικά στη συνάρτηση `generate()` με βάση περιορισμούς. Έχω δημιουργήσει με Photo editing το tree density mask, ώστε τα δέντρα να σχηματίζουν δαχτυλίδι γύρω από την λίμνη και ύστερα μερικά «stray δέντρα» εκτός δαχτυλιδιού. Με το `sampleMask(forestMask, ...)` γίνεται έλεγχος της μάσκας. Stray Trees: Στη συνάρτηση `isValidPlacement`, δέντρα εκτός του mask έχουν 2% πιθανότητα να τοποθετηθούν ("lucky stray trees").



Εύρεση Σωστού Ύψους (Heightmap Integration)

Για να εφάπτονται τα δέντρα σωστά στο έδαφος το σύστημα φορτώνει τα δυαδικά δεδομένα του terrain μέσω της `loadTerrainBinary`, αυτά έχουν παρχθεί από το Python script Με το Python script `heightmap_split.py` δημιουργείται το `terrain_data.bin` στο οποίο υπάρχει το heightmap (και οι μάσκες λίμνης, δάσους, κτλ) σε binary Μορφή ώστε να χρησιμοποιηθεί εύκολα από c++. Η συνάρτηση `sampleHeight(x, z)` υπολογίζει το ακριβές ύψος y στο grid, ώστε να μην αιωρούνται τα δέντρα.

Έλεγχος Ελεύθερων Θέσεων και Συγκρούσεων

Πριν την οριστική τοποθέτηση, εκτελούνται οι εξής έλεγχοι:

isValidPlacement: Ελέγχει αν το ύψος y είναι εντός των ορίων (`TREE_MIN_HEIGHT`, `TREE_MAX_HEIGHT`) και αν η θέση βρίσκεται μέσα σε λίμνη μέσω του `lakeMask`.

isFarFromExistingTrees: Εξασφαλίζει ότι τα δέντρα δεν αλληλοκαλύπτονται, ελέγχοντας την απόσταση τόσο από δέντρα του ίδιου του δάσους όσο και από εξωτερικά εμπόδια (π.χ. άλλα είδη δέντρων ή θάμνους) μέσω της `externalObstacles`. Αυτό είναι πολύ σημαντικό καθώς στο `winter.cpp` πρώτα εκτελείται η δημιουργία του δάσους των μηλιών και ύστερα των ελάτων/πεύκων. Οπότε πρέπει όλες οι θέσεις που έχουν πιάσει οι μηλιές να εξαιρεθούν.

Με την `addExternalPositions`, όλες οι πιασμένες θέσεις εξαιρούνται.

Καθώς οι θέσεις επιλέγονται τυχαία και μπορεί να μην είναι valid, δίνονται έξτρα προσπάθειες παραγωγής στην `generate()`, όπου έχω ορίσει `int maxAttempts = targetInstanceCount * 10;`

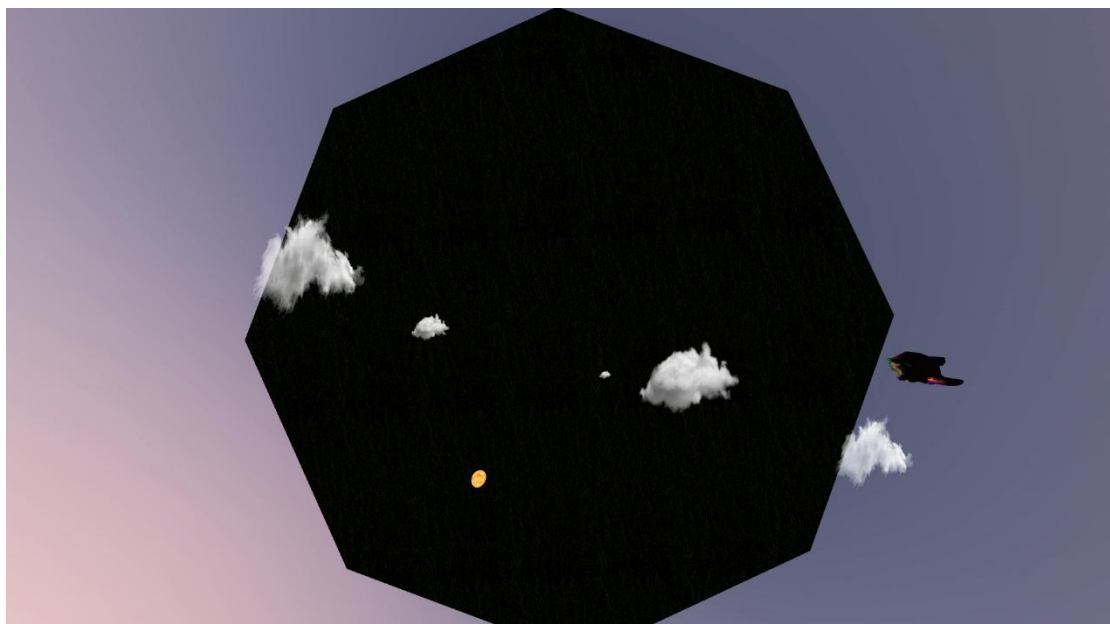
Παράγωγή στο create context:

```
// ===== APPLE & PINE FOREST =====  
// ===== APPLE FOREST =====  
// Apple Forest  
float scale = SCALING_FACTOR;  
// 1. Create Apple Forest (Mode 7, Scale 0.4)  
appleForest = new Forest(appleTreeModel, programs.lighting, 50, 0.7f, 7);  
appleForest->setTerrainBounds(  
    -scale / 2, scale / 2, // X bounds  
    -scale / 2, scale / 2, // Z bounds  
    0.0f, 50.0f, // Y bounds  
    1.0f // scaling factor ?? the heightmap is already scaled  
);  
appleForest->loadTerrainBinary("assets/heightmap/terrain_data.bin");  
appleForest->generate(); // Generate first  
  
// ===== PINE FOREST =====  
// pine tree  
pineForest = new Forest(pineTreeModel, programs.lighting, 50, 2.4f, 5);  
pineForest->setTerrainBounds(  
    -scale / 2, scale / 2, // X bounds  
    -scale / 2, scale / 2, // Z bounds  
    0.0f, 50.0f, // Y bounds  
    1.0f // scaling factor ?? the heightmap is already scaled  
);  
pineForest->loadTerrainBinary("assets/heightmap/terrain_data.bin");  
// 3. Tell Pine Forest where Apple Forest is to avoid overlap  
pineForest->addExternalPositions(appleForest->instances);  
pineForest->generate();
```

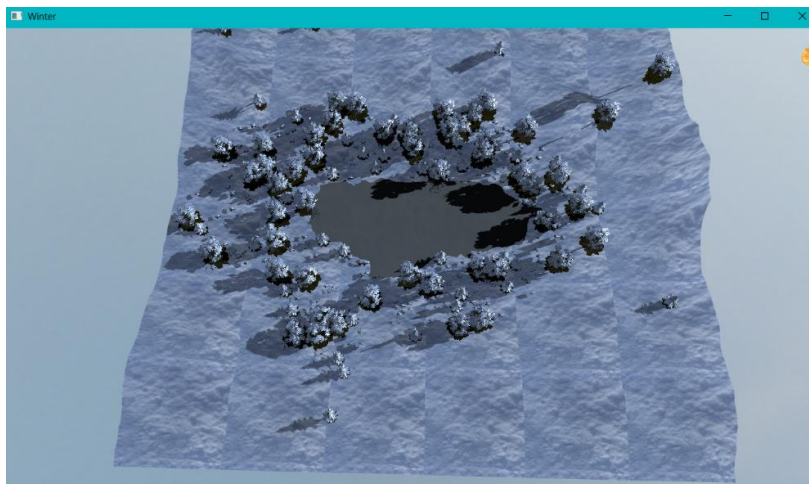
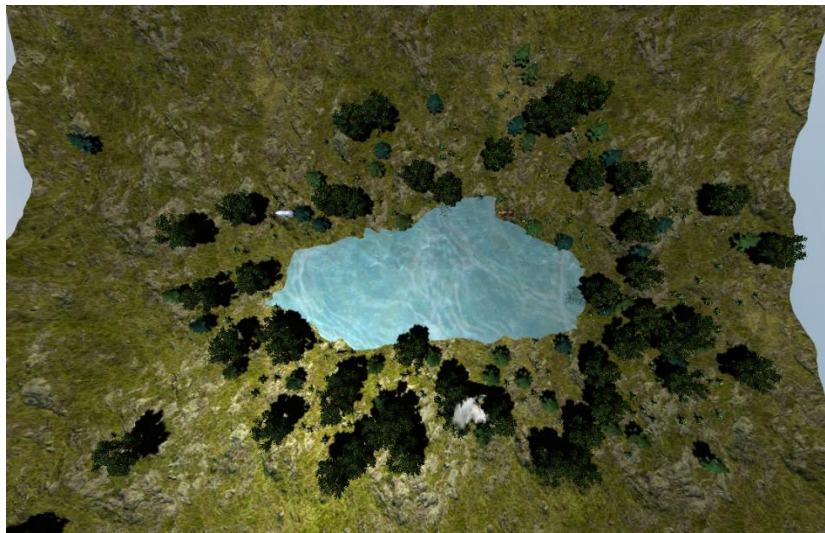
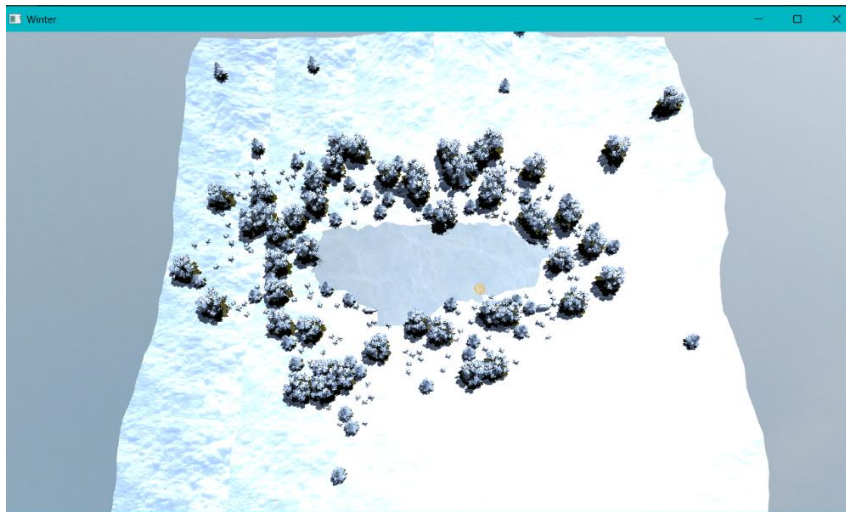
Bug & Επίλυση

Η αρχικοποίηση όλων των μεταβλητών που θα χρησιμοποιήσει ο shader έπρεπε να γίνουν από την createContext. Τα Instanced objects έχουν έξτρα μεταβλητές για την αποστολή του model matrix στον shader. Αν δεν αρχικοποιηθούν, η μνήμη γεμίζει με σκουπίδια.

```
glUseProgram(shaderProgram); //  
// THIS FIXED THE GIANT TREE ISSUE  
/* ===== VERY IMPORTANT ===== */  
useInstancingLocation=glGetUniformLocation(shaderProgram,"useInstancing");  
glUniform1i(useInstancingLocation, 0);  
// BEGIN WITH INSTANCING
```



Τελικό αποτέλεσμα:



Bush creation

Αντίστοιχα με το δάσος δημιουργήθηκε το `bush.cpp`. Οι θάμνοι δημιουργούνται μετά τα δάση οπότε με την συνάρτηση `setTreeReferences` προστίθενται όλες οι θέσεις των δέντρων που πρέπει να αποφευχθούν. Έλεγχοι με `isFarFromExistingBush`, `isFarFromTrees`, `isValidPlacement`. Η υπόλοιπη λογική παραμένει ίδια.

PART B

Snowfall

Για την προσομοίωση της χιονόπτωσης δημιουργήθηκε ένα **Particle System**, όπου τα particles αντιπροσωπεύουν τις νιφάδες που πέφτουν γύρω από τον παίκτη. Υλοποίηση στο **snow.cpp**

Κάθε νιφάδα ορίζεται από τη δομή SnowParticle, η οποία αποθηκεύει τη θέση, την ταχύτητα και το μέγεθός της.

Spawn γύρω από την κάμερα: Τα σωματίδια δημιουργούνται δυναμικά σε ένα εύρος 80 μονάδων γύρω από τη θέση της κάμερας. Έτσι, το χιόνι "ακολουθεί" τον παίκτη, δίνοντας την αίσθηση ότι ολόκληρος ο κόσμος χιονίζει χωρίς να χρειάζεται η επεξεργασία εκατομμυρίων σωματιδίων.

Ανακύκλωση (Reset): Όταν μια νιφάδα φτάσει στο έδαφος $y < 0$ ή απομακρυνθεί πολύ από την κάμερα, επανατοποθετείται στο ύψος spawnHeight με νέες τυχαίες παραμέτρους μέσω της resetParticle.

SnowSystem::resetParticle

```
// Spawn around camera position
p.position = glm::vec3(
    cameraPos.x + distX(gen),
    spawnHeight,
    cameraPos.z + distZ(gen));
```

Φυσική και Αέρα: Η κίνηση των σωματιδίων ενημερώνεται στη συνάρτηση update. Κάθε σωματίδιο έχει μια βασική κατακόρυφη ταχύτητα πτώσης. Η συνάρτηση update_velocity υλοποιεί την επίδραση του αέρα. Χρησιμοποιείται η μεταβλητή airResistance (0.7f), ώστε η ταχύτητα των νιφάδων να προσαρμόζεται ομαλά προς την κατεύθυνση του ανέμου (targetWindX/Z), αποφεύγοντας αφύσικες απότομες αλλαγές.

Αρχικά μόνο ελαφρύ drift:

```
// Falling velocity with slight horizontal drift
p.velocity = glm::vec3(
    (distX(gen) * 0.02f), // Slight X drift
    -distVel(gen),       // Falling speed
    (distZ(gen) * 0.02f) // Slight Z drift
);
```

Ώστερα στη SnowSystem::update_velocity, μετά την προσθήκη αέρα:

```
p.velocity.x += (targetWindX - p.velocity.x) * airResistance ;
p.velocity.z += (targetWindZ - p.velocity.z) * airResistance ;
```

όταν ο αέρας σταματάει η χιονόπτωση ξαναπέφτει κανονικά με κατεύθυνση μόνο προς τα κάτω.

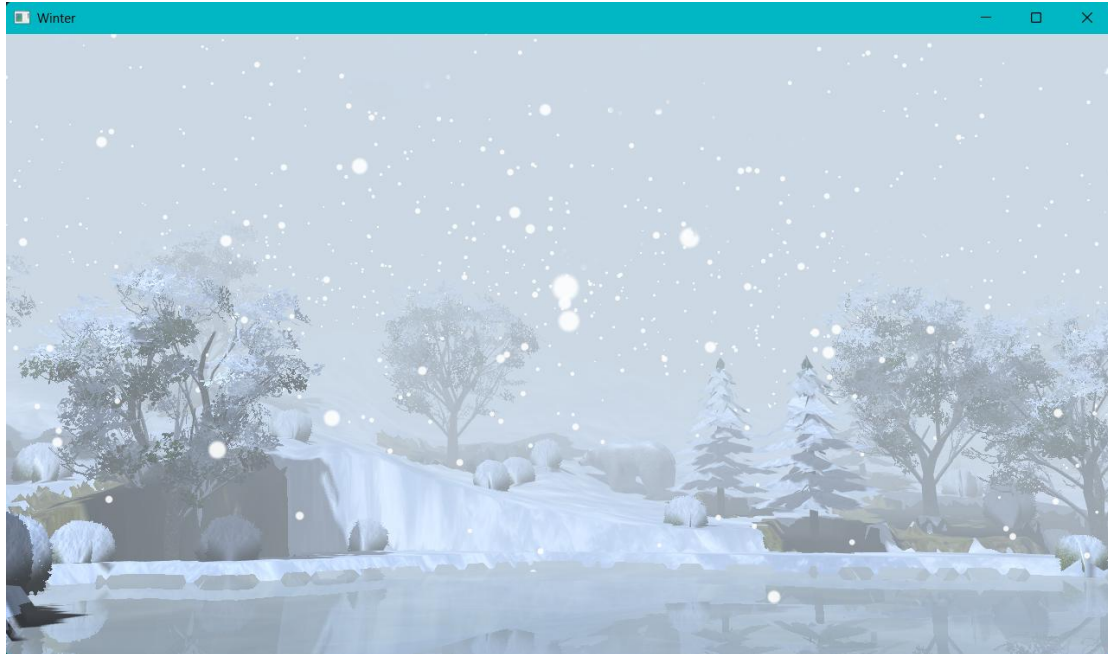
Rendering: Η σχεδίαση γίνεται με τη χρήση Point Sprites.

Vertex Shader: Υπολογίζει τη θέση στον κόσμο και κλιμακώνει το μέγεθος της νιφάδας (gl_PointSize) με βάση την απόσταση από την κάμερα, ώστε οι κοντινές νιφάδες να φαίνονται μεγαλύτερες.

Fragment Shader: Αντί για φόρτωση υφής, η κυκλική μορφή της νιφάδας παράγεται μαθηματικά. Υπολογίζεται η απόσταση από το κέντρο του point (gl_PointCoord) και απορρίπτονται τα pixels εκτός ακτίνας 0.5.

Smooth Edges: Χρησιμοποιείται η συνάρτηση smoothstep για να δημιουργηθούν "μαλακές" άκρες με διαφάνεια (alpha blending), κάνοντας το χιόνι να φαίνεται πιο ρεαλιστικό και λιγότερο "ψηφιακό".

Το σύστημα ενεργοποιείται και απενεργοποιείται μέσω της toggle(), η οποία αρχικοποιεί όλα τα σωματίδια όταν ξεκινά η χιονόπτωση.



Fog

Στον fragment shader γίνεται ανάμιξη του χρώματος του fragment με το γκρι της ομίχλης:

```
uniform float fogDensity; //=1.0; // 0.0 to 1.0
uniform vec3 fogColor; // = vec3(0.8, 0.85, 0.9);;
```

Στον ουρανό, για να γίνει γκρι όλη η υφή του skydome ανακατεύεται για όλα τα fragments το χρώμα τους με το γκρι, έτσι ώστε όταν φτάσει fog factor=1 στο winter.cpp να γίνεται όλο το background γκρι. (σαν να είχα θέσει το gl_clear σε 0.8, 0.85, 0.9)

```
fragmentColor = vec4(texture(skyTex, vec2(u, v)).rgb, 1.0);
fragmentColor.rgb = mix(fragmentColor.rgb, fogColor, fogDensity );
```

Για τα υπόλοιπα αντικείμενα ακολουθείται η εξής λογική έτσι ώστε αυτά που είναι πιο κοντά να φαίνονται καλύτερα (λιγότερη ανάμιξη με γκρι).

```
float dist = length(vertex_position_cameraspace.xyz);
float fogDensityScaled = fogDensity * 0.03;
float fogFactor = 1.0 / exp((dist * fogDensityScaled) * (dist * fogDensityScaled));
fogFactor = clamp(fogFactor, 0.0, 1.0);
fragmentColor.rgb = mix(fogColor, fragmentColor.rgb, fogFactor);
```

<https://www.youtube.com/watch?v=oQksg57qsRA>

Snow accumulation

Θέλουμε το τοπίο να γίνεται χιονισμένο. Για αυτό θα εφαρμοστούν τρία στάδια:

1. Χιονονιφάδες
2. Εφαρμογή snowy texture στο έδαφος
3. Inflation του εδάφους, «φούσκωμα»

Snowflakes

Για να φαίνεται σαν τα αντικείμενα να λερώνονται με χιονονιφάδες, έχω δημιουργήσει στον shader την προσθήκη μαθηματικά υπολογισμένου θορύβου. `// snowflakes +snow`
Για καλύτερη απόδοση, χιονονιφάδες εμφανίζονται μόνο όταν είμαστε κοντά σε κάποιο αντικείμενο, δηλαδή `detailFade > 0.01`, όπου:

```
float distToCamera = length(vertex_position_cameraspace.xyz);  
float detailFade = smoothstep(50.0, 10.0, distToCamera);
```

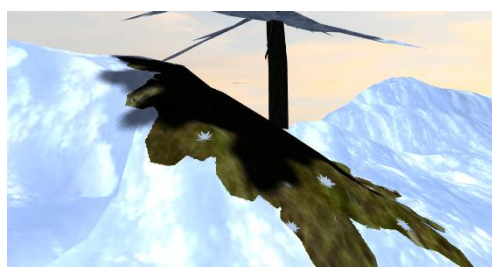
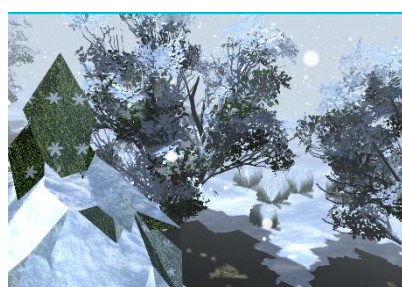
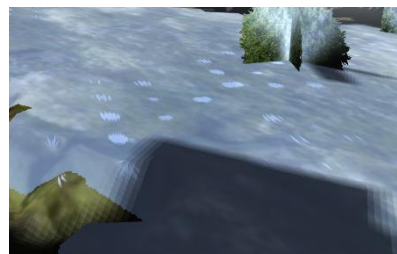
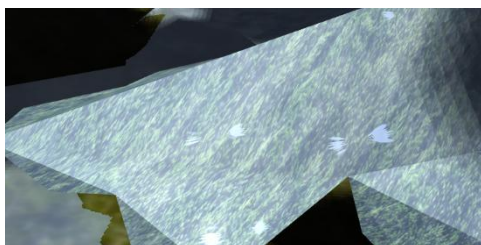
Οι χιονονιφάδες θεωρούν τα αντικείμενα στα οποία πέφτουν grid και επειδή θέλω να είναι πιο αραιές και μεγάλες στα δέντρα σε σχέση με το terrain, οπότε έτσι γίνεται η εξής ανάθεση scale factor.

```
float gridSize = 2.0;  
if (useTexture == 5 || useTexture == 6 || useTexture == 7 ){  
    gridSize = 0.1; // sparser flakes  
}
```

Έτσι ώστε να μην πέφτουν όλες ταυτόχρονα προστίθεται `delay birthThreshold = hash(cellID)` και η χιονονιφάδα πέφτει μόνο όταν `if (birthThreshold < snowAmount)`. Επιπλέον στο `snowAccumulationTime` της `winter.cpp` ξεκινάει από -4 δίνοντας χρόνο στο χιόνι να φτάσει πρώτα στο έδαφος πριν αρχίσουν να εμφανίζονται χιονιάδες και λοιπά εφέ.

Υπολογίζω αν η επιφάνεια του αντικειμένου κοιτάει προς τα πάνω (εσωτερικό γινόμενο normal και 0,1,0). Αν ναι εμφανίζω χιονονιφάδα με «ακανόνιστο σχήμα».

```
// Define shape  
float angle = atan(cellUV.y, cellUV.x);  
float starMod = abs(cos(angle * 3.0) + cos(angle * 13));  
float snowflakeDist = dist / (1.0 + 0.4 * starMod);  
float flakeSize = 0.1 + (hash(cellID + 1.0) * 0.1);  
float shape = smoothstep(flakeSize, flakeSize - 0.05, snowflakeDist);  
vec3 flakeColor = vec3(0.85, 0.9, 1.0);  
float finalAlpha = shape * detailFade;  
_Kd.rgb = mix(_Kd.rgb, flakeColor, finalAlpha);
```



Snow Texture

Όταν το επίπεδο του χιονιού περάσει ένα συγκεκριμένο threshold (snowLevel στο winter.cpp και snowAmount στον fragmentshader) θέλουμε το έδαφος να καλυφθεί με χιόνι. Για αυτό γίνονται δυο έλεγχοι:

1. Φαίνεται το έδαφος από τον ουρανό; Πηγή χιονιού
2. Πόσο απότομη είναι η επιφάνεια του αντικειμένου;

Δημιουργείται μια ορθογραφική κάμερα πάνω από την σκηνή (`mat4 getSkyViewMatrix()` στο `winter.cpp`). Η συνάρτηση `sky_visibility_pass()` στο `winter.cpp` σχηματίζει ένα 2D depth map από την οπτική γωνία του ουρανού (όπως ο χάρτης σκιών). Λευκές περιοχές φαίνονται, επιτρέπουν χιόνι, σκοτεινές περιοχές μπλοκάρονται από κάποιο αντικείμενο, δεν πέφτει εκεί χιόνι. Στη συνέχεια, κάθε fragment θα μεταφερθεί σε sky space προκειμένου να αποφασιστεί αν πρέπει να καλυφθεί από χιόνι.

```
// Transform fragment position to sky-space
vec4 fragPosSky = skyVP * vec4(vertex_position_worldspace, 1.0);
vec3 skyCoords = (fragPosSky.xyz / fragPosSky.w) * 0.5 + 0.5;

// Check if this fragment is visible from the sky // (comparing depth
values like shadow mapping)
float skyDepth = texture(snowAccumMap, skyCoords.xy).r;
float fragDepth = skyCoords.z;
float skyVisible = (fragDepth - 0.001) < skyDepth ? 1.0 : 0.0; // Small bias
```

ΚΑΙΎστερα γίνεται έλεγχος των Normals. Το χιόνι αρχίζει σε επιφάνειες με κλίση $\cos=0.3$ και είναι πλήρες στις επίπεδες επιφάνειες. ο ο χρόνος περνάει η υφή γίνεται πιο έντονη:

```
// Calculate final snow coverage
float snowCoverage = skyVisible * normalFactor * snowAmount;
```

Snow Inflation

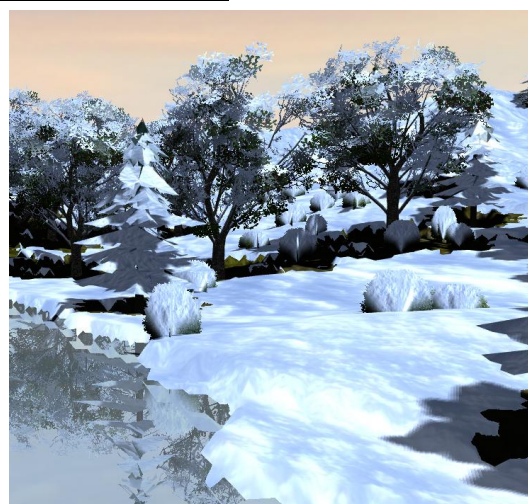
Στον vertex shader φροντίζουμε όσο περνάει ο χρόνος και ανεβαίνει το snow amount το έδαφος να φουσκώνει. Αν το vertex φαίνεται από τον ουρανό σημαίνει ότι θα πέσει πάνω του χιόνι, οπότε θα πρέπει να ανασηκωθεί:

```
float skyDepth = texture(snowAccumMap, skyCoords.xy).r;
float fragDepth = skyCoords.z;
// Is this vertex hit by snow?
float skyVisible = (fragDepth - 0.01) < skyDepth ? 1.0 : 0.0;
```

Όπως και προηγουμένως, ελέγχουμε τα normal. Απότομες πλαγιές πρέπει να φουσκώσουν λιγότερο. **Οπότε στο τέλος υπολογίζουμε ένα βάρος ανάλογο του skyvisibility, της κλίσης και της ποσότητας χιονιού.**

```
float weight = skyVisible * normalFactor * snowAmount;
```

Στη συνέχεια μετακινούμε τα vertices προς τα πάνω `worldPos.y += inflationAmount;`



Frozen lake + Reflection

Reflection

Προσθήκη αντανάκλασης για καλύτερο ρεαλισμό. Όταν η λίμνη παγώνει αυξάνουμε reflection factor. Προσθήκη δύο νέων συναρτήσεων στο camera.cpp

```
mat4 getReflectionMatrix, // Main reflection function
```

```
mat4 Camera::getReflectionViewMatrix(float waterHeight)
```

Ορίζουμε PlaneNormal και φτιάχνουμε το reflection matrix έτσι ώστε:

$newY = -oldY + 2 \times waterHeight$, ενώ X,Z μένουν ίδια.

Στη συνέχεια με την `getReflectionViewMatrix`, μεταφέρουμε το Camera matrix σε Mirrored Position.

Στο winter.cpp με την `reflection_pass` κάνουμε render όλη την σκηνή, ΕΚΤΟΣ της λίμνης (δεν γίνεται να ανακλάται στον εαυτό της) από την οπτική γωνία της mirrored κάμερας. Αυτό αποθηκεύεται στο `reflectionTexture`, το οποίο θα ζωγραφιστεί πάνω στη λίμνη. Με την συνάρτηση `createReflectionFBO` φτιάχνουμε έναν frame buffer όπου θα αποθηκευτεί η εικόνα αυτή της αντανάκλασης. Επομένως η εικόνα που θα υπολογιστεί από το `reflectionpass` θα περαστεί στο `reflectionTexture`.

```
// 3. ΚΛΕΙΔΙ: Attach to texture στο FBO
```

```
glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0,  
                        GL_TEXTURE_2D, reflectionTexture, 0);
```

```
void reflection_pass(mat4 reflectionView, mat4 projectionMatrix) {  
    int fb_width, fb_height;  
    glfwGetFramebufferSize(window, &fb_width, &fb_height);  
    render_scene(reflectionView, projectionMatrix, fb_width, fb_height, 0.0f, RenderMode::REFLECTION);  
}
```

Στον fragment shader φροντίζουμε όταν γίνεται υπολογισμός του reflection να μην ζωγραφίζονται water pixels

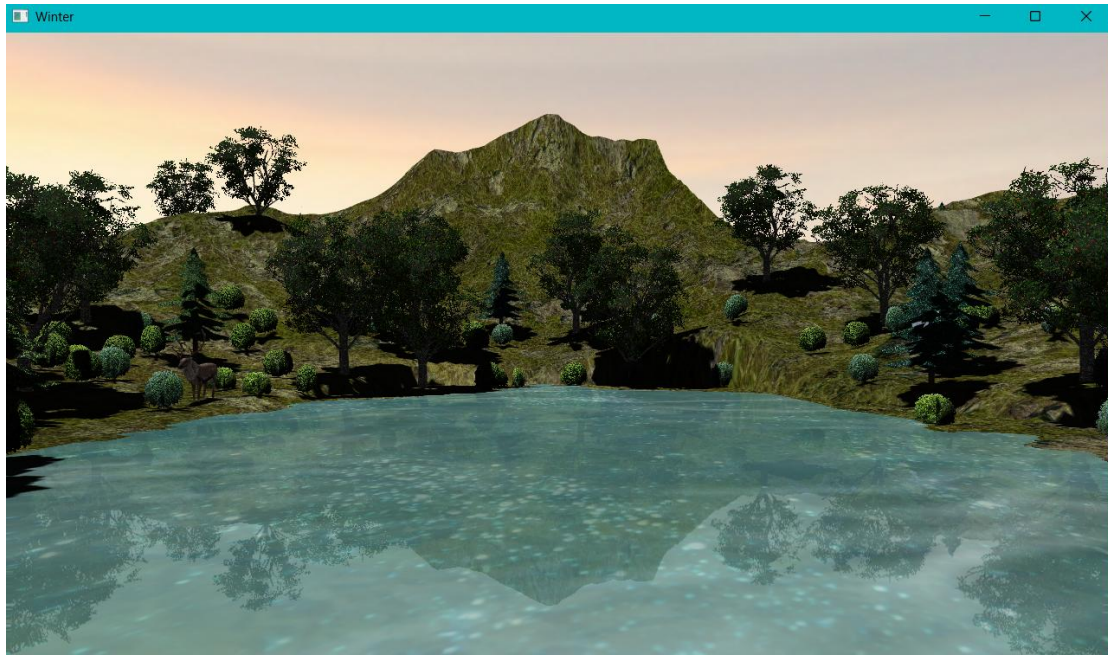
```
// NEW: During reflection pass, discard water pixels  
if (renderingReflection == 1 && lakeMask > 0.5) {  
    discard; // Don't render water in reflection  
}
```

Επίσης στο τέλος αναμειγνύουμε την υφή του νερού με την αντανάκλαση (reflection color) ανάλογα με το `reflectionStrength`, μια μεταβλητή ελεγχόμενη από την cpu (0.2 για κινούμενο νερό, 0.8 για ακίνητο νερό).

```
finalWater = mix(finalWater, reflectionColor, reflectionStrength);
```

Επιπλέον όταν το νερό δεν είναι παγωμένο εφαρμόζεται μια έξτρα παραμόρφωση στις αντανάκλασεις, σαν τα κύματα να μετακινούν τα αντανακλώμενα δέντρα. Το `waterFlow` χρησιμοποιείται και στην μετακίνηση των υν του νερού, οπότε δίνει μια μικρή ομοιότητα.

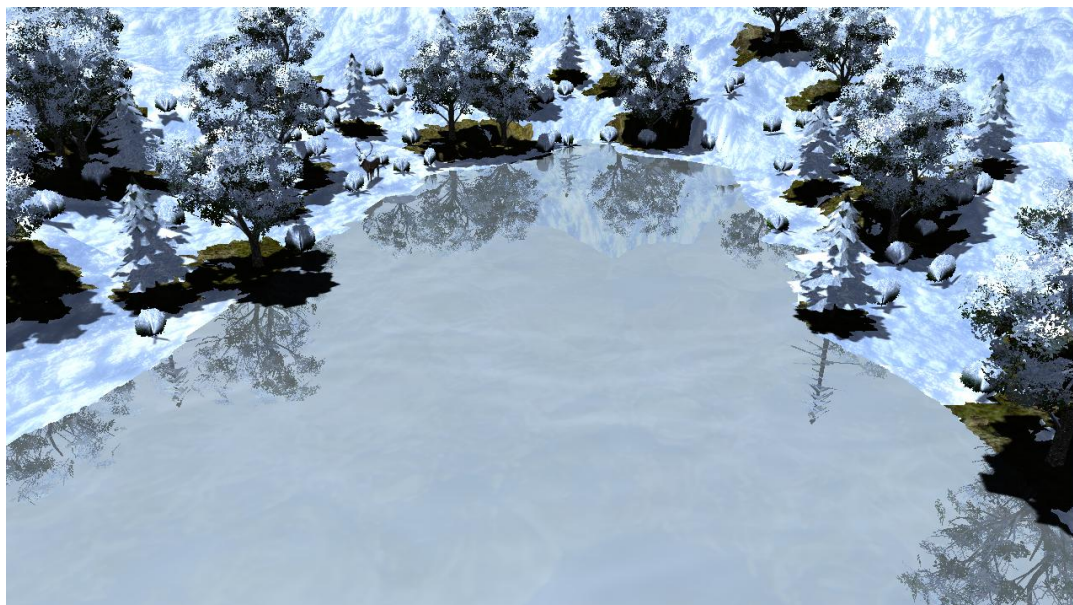
```
if(freezeFactor<0.5){  
    // Add wave distortion to reflection based on water animation  
    float wave1 = sin(time * 0.4 + (vertex_UV.x + waterFlow.x) * 8.0);  
    float wave2 = cos(time * 0.3 + (vertex_UV.y + waterFlow.y) * 6.0);  
  
    vec2 waveDistortion = vec2(wave1, wave2) * 0.0006;  
}
```



Frozen Lake

Όταν το `freezeFactor` μεγαλώσει (>0.85) η υφή του νερού σταματάει να κουνιέται. Κρατάω την εικόνα του εργαστηρίου γιατί σχηματίζει όμορφες ανεπαίσθητες αυλακώσεις πάνω στον πάγο. Αν το `freezeFactor` >0 και μικρότερο του 0.85 δεν γίνεται animation αλλά το χρώμα είναι ανάμιξη των 2 `waterTex` για ομαλή μετάβαση.

Στη συνέχεια αναμειγνύουμε το χρώμα του fragment με χρώμα πάγου (`frozenColor = vec3(0.85, 0.90, 0.95); // Icy blue-white color`), αφού Μειώσουμε τον κορεσμό saturation του αρχικού χρώματος. Στη συνέχεια για προσθήκη «θορύβου» που θα κάνει τον πάγο πιο ρεαλιστικό, χρησιμοποίησα το `NOISETEX`, δηλαδή μία εικόνα `Voronoi`. Το χρώμα είναι ανάλογο της απόστασης από τα σημεία, και οι γραμμές του γράφου βορονόι είναι εντελώς λευκές ενώ τα κέντρα μαύρα. Αυτή η εικόνα έχει χρησιμοποιηθεί σαν μάσκα, και στα λευκότερα σημεία της τοποθετείται λευκό χρώμα ενώ στα υπόλοιπα της παγωμένης λιμνης που υπολογίσαμε πιο πριν.



6. Wind

Όπως αναλύθηκε στο χιόνι, η ταχύτητα του αέρα λαμβάνεται υπόψιν από τις χιονονιφάδες, επιταχύνοντας τες προς μία κατεύθυνση.

Για τα δέντρα στον vertex shader κουνιούνται τα vertices με ελαφρώς ψευδοτυχαία κίνηση χρησιμοποιώντας sin και cos. Όμως η κατεύθυνση του αέρα λαμβάνεται υπόψιν.

Το V ενεργοποιεί τον αέρα και τα BNM, αλλάζουν το x_bias και z_bias, δηλαδή πόσο φυσάει κατά z και πόσο κατά x.

```
// ===== WIND LOGIC στον vertex shader =====
if(windStrength>0){
    float phaseOffset = modelMatrix[3].x + modelMatrix[3].z; //unique για
ka8e object
    float swayFactor = pos.y * float(windStrength) * 0.01; // oso pio
pshla toso pio entono
    float wave_x = sin(time * 2.0 + phaseOffset) * (1.0+x_bias) ;
    float wave_z = cos(time * 1.5 + phaseOffset*1.5) * (1.0+z_bias) ;
    if (useInstancing){
        // Apply displacement to X and Z coordinates
        pos.x += wave_x * swayFactor;
        pos.z += wave_z * swayFactor ;
    }
}
```

Μετακινούμε δηλαδή τα vertices κατά x και κατά z ανάλογα με τη κατεύθυνση του αέρα αλλά και με έξτρα τυχειότητα, δημιουργώντας μια ρεαλιστική κίνηση οπτικά.



7. a. Clouds clear away

Η ομίχλη αραιώνει με τον ίδιο μηχανισμό με τον οποίο εμφανίστηκε. Τα σύννεφα διαγράφονται όσο υπάρχει ακόμα ομίχλη. Δηλαδή με δεύτερο πάτημα του G το χιόνι σταματάει και η ομίχλη μειώνεται.

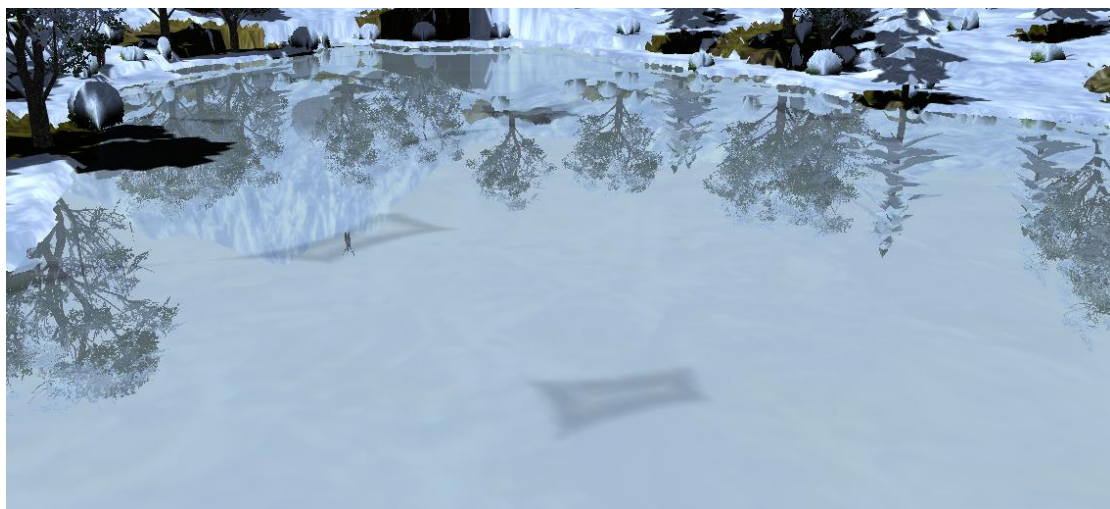
Πατώντας παρατεταμένα το κουμπί T, το περιβάλλον αρχίζει να λιώνει.

b. ice crack lake melts

Sound effect λίμνης που σπάει και χρήση Voronoi texture για ρεαλιστικό λιώσιμο της λίμνης. Τα white edges θα σπάσουν πρώτα, θα αποκτήσουν δηλαδή πιο σκούρο χρώμα. Όσο ο χρόνος περνάει meltfactor αυξάνεται, επομένως μεγαλύτερο κομμάτι της μάσκας voronoi θα περνάει το threshold και μεγαλύτερο κομμάτι θα έχει πιο σκούρο χρώμα. Αύξησα το contrast της εικόνας ώστε να είναι πιο απότομα τα αρχικά cracks.

Ύστερα από ένα σημείο θα ξαναξεκινήσει το animation του νερού και στο τέλος θα μείνουν μόνο λευκά κομμάτι πάγου πριν εξαφανιστούν.

```
// --- if melting => ICE CRACKS ---
if(meltFactor > 0.01 && meltFactor<1 ) {
    float voronoi = texture(noiseTex, vertex_UV * 6.0).r;
    float crackThreshold = 0.9 - (meltFactor * 0.7); // Starts at 0.9,
    goes down to 0.2
    float cracks = smoothstep(crackThreshold, crackThreshold + 0.1,
    voronoi);
    // 2. Define Ice and Crack Colors
    vec3 iceColor = vec3(0.82, 0.90, 0.95); // Light bluish-white ice
    vec3 crackColor = vec3(0.05, 0.13, 0.23); // Dark blue-gray water in
    cracks
    // 3. Mix ice with cracks (cracks = 1.0 means show crack color)
    vec3 crackedIce = mix(iceColor, crackColor, cracks);
    // 4. The "Melting" Alpha
    // Use INVERTED voronoi: ice chunks (dark centers) disappear first
    float iceVisibility = smoothstep(meltFactor - 0.1, meltFactor + 0.1,
    1.0 - voronoi);
```





Σημείωση: η εικόνα voronoi έδωσε πιο ρεαλιστικό αποτέλεσμα από Worley noise και απλό θόρυβο.

c. snow melts

Για να λιώνει το χιόνι και να μην γίνεται απλώς fade out με τη μείωση του snow amount, έχει χρησιμοποιηθεί η ίδια εικόνα voronoi σαν μάσκα, όμως στις λευκές άκρες αυτή τη φορά απλά γίνεται πιο διαφανής η εικόνα του χιονιού μέχρι να εξαφανιστεί τελείως.

```
// MELTING LOGIC
if(meltFactor>0.01){
    // Use noise to create "patchy" melting
    float noise = texture(noiseTex, vertex_UV_tiled * 4.0).r;
    //float meltThreshold = 1 - (meltFactor * 1); // Starts at 1, goes
down to 0.0
    float slowFactor = pow(meltFactor, 2.0); // try 2.0, 3.0, 4.0
    float meltThreshold = 1.0 - slowFactor;
    float meltedSnow = smoothstep(meltThreshold, meltThreshold + 0.1,
noise);
    //float patchyCoverage =snowCoverage-meltedSnow;
```

```

    patchyCoverage = snowCoverage * (1.0 - meltedSnow);}
else{
    patchyCoverage = snowCoverage; //NOTHING MELTED
}

```

Παράλληλα τα vertices μετακινούνται με τον εξής τρόπο ώστε να προσομοιώνουν το rolling downhill motion:

Διάνυσμα βαρύτητας σε worldspace (0,-1,0).

Πώς βρίσκουμε διάνυσμα παράλληλο στην επιφάνεια που δείχνει το downhill motion?

Σπάω το g σε κάθετη συνιστώσα και παράλληλη συνιστώσα σε σλεχση με διεύθυνση Normals. $\cos\theta = -N \cdot g$ άρα $\text{vec3 downhill} = g - \text{dot}(g, n) * n$;

Επιπλέον βρίσκουμε πόσο απότομη είναι η πλαγιά $\text{steepness} = 1.0 - \text{dot}(n, \text{vec3}(0,1,0))$;

Με βάση αυτούς τους δύο παράγοντες βρίσκουμε πόσο χιόνι θα κυλήσει και κάνουμε update to worldPos.

```

float slideAmount = meltFactor * steepness * 0.4;
worldPos.xyz += downhill * slideAmount;

```

Για το γέμισμα της λίμνης ανασηκώνουμε ελαφρώς τα vertices της λίμνης

```

if(maskValue>0.5 && meltFactor>0.01 && meltFactor<1){
    worldPos.y += 0.1*meltFactor;
}

```

Όταν το meltfactor πάει στο 0 η λίμνη επιστρέφει στην αρχική της ανοιξιάτικη μορφή (τα vertices ξαναπέφτουν, η θερμοκρασία γύρισε στο αρχικό).

d. avalanche

Δημιουργία του AvalancheBall.cpp

spawn(pos): Δημιουργεί μια σφαίρα σε συγκεκριμένο σημείο (υψηλά στο βουνό), μηδενίζει την ταχύτητα και ορίζει την αρχική ακτίνα 0.5.

update(deltaTime): Χρησιμοποιεί τη `sampleHeightAt` σε 4 σημεία γύρω από τη σφαίρα για να βρει το διάνυσμα της κατηφόρας (`slopeDir`). Προσθέτει επιτάχυνση στην κατεύθυνση της κλίσης (Gravity) και εφαρμόζει τριβή (0.99f). Όσο η σφαίρα κυλάει στο έδαφος με ταχύτητα, η ακτίνα (`radius`) αυξάνεται.

Το **λιώσιμο** ενεργοποιείται όταν η σφαίρα μπει στο νερό (`inWater`) ή κολλήσει στο έδαφος (`isStuck`).

Μηχανισμός Λιωσίματος: Χρησιμοποιείται ένας χρονοδιακόπτης (`meltTimer`). Ο συντελεστής `meltFactor` (από 0 έως 1) μειώνει σταδιακά την ακτίνα μέχρι η σφαίρα να εξαφανιστεί (`active = false`). Επιπλέον για να υπάρχει παραμόρφωση όσο λιώνει (`squashing`), Στη συνάρτηση `getModelMatrix()`, όταν η σφαίρα λιώνει, δεν μικραίνει ομοιόμορφα:

Άξονας Y (Υψος): Πολλαπλασιάζεται με έναν παράγοντα που μειώνεται (έως 0.3), κάνοντας τη σφαίρα να "κάθεται" ή να πλαταίνει.

Άξονες X και Z (Πλάτος): Πολλαπλασιάζονται με έναν παράγοντα που αυξάνεται (έως 1.3), προσομοιώνοντας την εξάπλωση του χιονιού που λιώνει στο έδαφος ή το νερό.

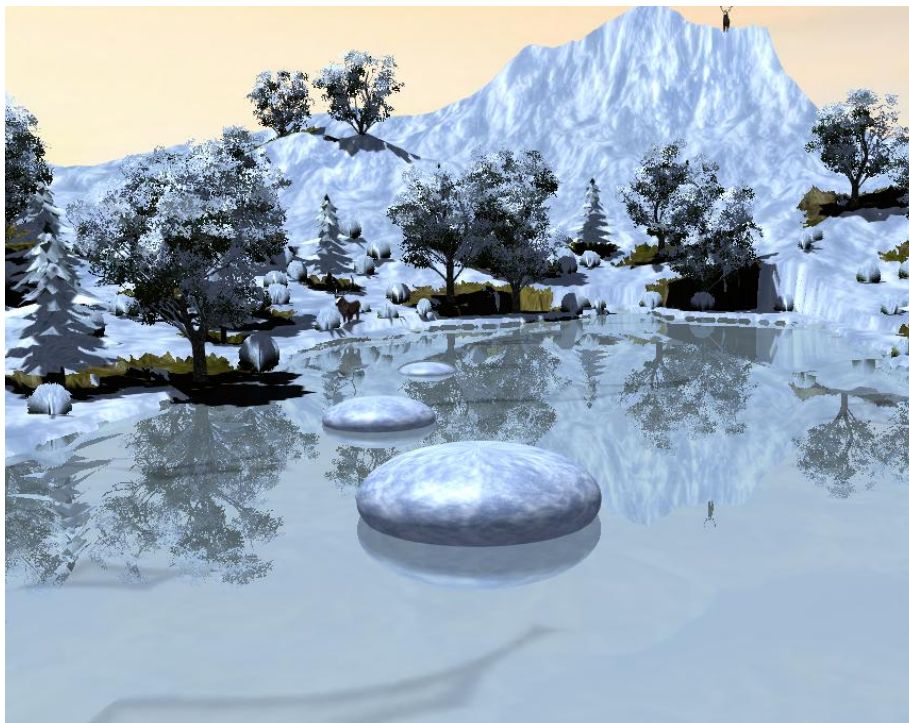
Έλεγχος και Ήχος (`winter.cpp`)

Ενεργοποίηση: Η χιονοστιβάδα ξεκινά μόνο όταν ο χρήστης πατήσει το πλήκτρο 'T' και περάσουν μερικά δευτερόλεπτα. Μόλις το επίπεδο λιωσίματος (`meltLevel`) ξεπεράσει το 0.1, δημιουργούνται 3 σφαίρες σε τυχαίες θέσεις ψηλά στο βουνό.

Ήχος:

Μόλις πατηθεί το 'T', παίζει ο ήχος `ice_crack` (σπάσιμο πάγου).

Αφού περάσουν 2 δευτερόλεπτα από την ενεργοποίηση της χιονοστιβάδας, ξεκινά ο ήχος `avalanche`.



Miscellaneous

Controls παιχνιδιού

Ο παίχτης του παιχνιδιού είναι ουσιαστικά σα μάγος που ελέγχει τον καιρό· μπορεί να περπατήσει, να πετάξει, να τηλεμεταφερθεί ψηλά για να δει την θέα, να συρρικνώσει το ελάφι, να αρχίσει την χιονόπτωση και το λιώσιμο του χιονιού. Όλα τα controls υλοποιούνται στην mainloop.

AWSD keys: κίνηση

Ποντίκι: κάμερα

Βελάκια: Zoom in, zoom out (FoV)

IJKL: αλλαγή κατεύθυνσης ήλιου

V: ενεργοποίηση αέρα

BNM,: έλεγχος έντασης αέρα στις κατευθύνσεις z,x

G: χιονόπτωση ξεκινά

H: max επίπεδο χιονιού

T: κρατώντας παρατεταμένα το T λιώνει το χιόνι. Την πρώτη φορά δημιουργείται χιονοστιβάδα

F: ο παίχτης κάνει toggle μεταξύ flying και walking mode, δηλαδή να είναι περιορισμένος πανω στο terrain ή να κινείται ελεύθερα

Q: τηλεμεταφορά στο 0,50,0 για να δεις τη θέα, ΑΝ έχει ενεργοποιηθεί το flying mode

E: συρρικνώνει το ελάφι δίπλα στη λίμνη.

Flying vs Walking mode F key

Όταν το F έχει πατηθεί, η θέση της κάμερα γ περιορίζεται 1.8 μονάδες πάνω από το terrain. Επίσης ανάλογα το σημείο στο οποίο περπατάει ο παίκτης ακούγεται το αντίστοιχο sfx. Όταν ξαναπατηθεί το F μπαίνει σε flight mode και κινείται ελεύθερα στο χώρο.

Με το Q γίνεται τηλεμεταφορά (με sound effect) 0,50,0 για να δει τη θέα από ψηλά, μόνο εφόσον είναι σε flying mode.

Υλοποίηση:

Με το Python script **heightmap_split.py δημιουργείται το** terrain_data.bin στο οποίο υπάρχει το heightmap (και οι μάσκες λίμνης, δάσους, κτλ) σε binary Μορφή ώστε να χρησιμοποιηθεί εύκολα από c++. Η συνάρτηση getHeightDataOnly του winter.cpp φορτώνει μόνο τα δεδομένα ύψους που χρειάζονται από την main. Με την **float** sampleHeightAt για κάθε x,z σημείο γνωρίζω το γ του και έτσι τοποθετώ τα ζώα στα επιθυμητά x,y και ύστερα δημιουργώ την **void** constrainCameraToTerrain που ελέγχει την απόσταση.

Footsteps

struct FootstepSystem

```
vec3 lastPosition = vec3(0.0f);  
bool isMoving  
enum SurfaceType  
bool shouldPlayStep()
```

`void` update

Δημιουργώ την δομή δεδομένων FootstepSystem μέσα στο winter.cpp. Συγκρίνω την τωρινή θέση με την προηγούμενη έτσι ώστε να παίζεται ο ήχος μόνο όταν ο παίκτης κουνιέται (`bool shouldPlayStep()`). Με το enum Surface type, ορίζω 4 διαφορετικές επιφάνειες: χιόνι, έδαφος, νερό, πάγος. Αν το ύψος του terrain είναι ίσο με το waterHeight τότε είμαστε την λίμνη, αλλιώς είμαστε το έδαφος. Αν έχει snowAmount>0.6 τότε είμαστε σε χιόνι/ πάγο και όχι σε έδαφος/νερό.

Interactions με ζώα

SFX:

Αν ο παίχτης πάει πολύ κοντά σε κάποια από τις δύο αρκούδες, θα αρχίσουν να γρυλίζουν. Μόλις απομακρυνθεί από την ακτίνα τους, θα πάψει το γρύλισμα.

Αν ο παίχτης πάει πολύ κοντά στον λύκο θα αρχίσει να αλυχτάει. Πάλι αν απομακρυνθεί ο ήχος σταματάει.

Τα παραπάνω έγιναν με χρήση της `miniaudio.h`.

Στο αρχείο `AudioManager.h` λειτουργεί σαν wrapper, έχοντας τις συναρτήσεις `preload`, `playPreloaded`, `stopPreloaded`, έτσι ώστε να φορτώνει τα mp3 πριν αρχίσει το παιχνίδι, να αρχίζει την αναπαραγωγή τους και να την σταματάει, αντίστοιχα.

Τα ζώα είναι δωρεάν assets. Το ελάφι είχε ασπρόμαυρο texture στο blender, το οποίο έκανα export και ύστερα με photo editing καφέ. Επίσης η αρκούδα ήταν πολική (είχε λευκή υφή), οπότε στο παιχνίδι έχω δύο αρκούδες, μία καφέ (edited) και μία πολική.



Αναπνοή:

Όλα τα ζώα **αναπνέουν**. Αυτό έχει επιτευχθεί ως εξής στον vertex shader:

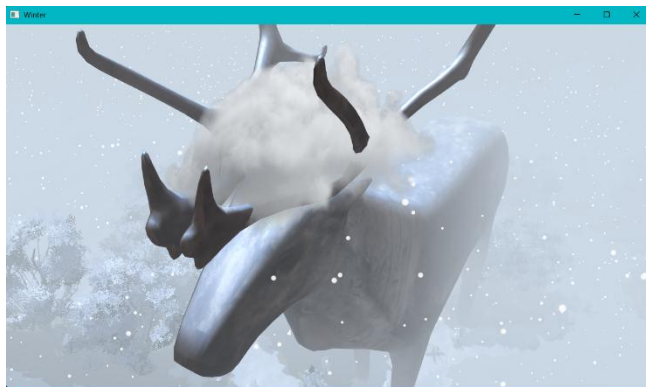
```
//make animals breathe
if (useTexture==7 && !useInstancing){
    float breatheFactor = pos.y * 0.01; // oso pio pshla toso pio entono
    float breathe = sin(time *3.0) *breatheFactor* 0.5;
    pos.y += breathe;}
```

Δηλαδή η γ συντεταγμένη τους μεταβάλλεται σαν ημίτονο. Όσο πιο ψηλά στο σώμα είναι, τόσο περισσότερο μεταβάλλεται, έτσι τα πόδια μοιάζουν ακίνητα ενώ ο κορμός ανεβοκατεβαίνει.

Ελάφι:

Καθώς δεν βρήκα ικανοποιητικό sound effect για το ελάφι, αποφάσισα να του δώσω κάποιες μαγικές ιδιότητες. Όταν ο παίκτης πάει πολύ κοντά του αυτό αρχίζει να μεγαλώνει (scale αν η κάμερα σε ακτίνα 3.0). Όταν ο παίκτης βγει εκτός ακτίνας σταματάει να μεγαλώνει. Αν πάει πολύ μακριά (33 μονάδες) το ελάφι γυρνάει στο φυσιολογικό του μέγεθος

Θέλοντας να δώσω στον παίκτη τη δυνατότητα να αντισταθεί σε αυτόν τον εκφοβισμό, πατώντας Ε μπορεί να μειώσει το scale του ελαφιού. Ταυτόχρονα παίζει ένα sound effect μαγικού spell.



Σύννεφο περνάει πάνω από το scaled up ελάφι.



Full screen mode

```
#define FULL_SCREEN 0 ή 1
```

Σημείωση: πώς να γίνει η εφαρμογή full screen

Στην initialize:

```
GLFWmonitor* primary_monitor = glfwGetPrimaryMonitor();
const GLFWvidmode* mode = glfwGetVideoMode(primary_monitor);
window = glfwCreateWindow(mode->width, mode->height, TITLE,
primary_monitor, NULL);

if (window == NULL) {
    glfwTerminate();
    throw runtime_error(string(string("Failed to open GLFW
window.") +
        " If you have an Intel GPU, they are not 3.3 compatible." +
        "Try the 2.1 version.\n"));
}
```

Στη mainloop:

```
// get screen size
int fb_width, fb_height;
glfwGetFramebufferSize(window, &fb_width, &fb_height);
```

Περνάμε στην κάμερα το νέο aspect ratio.

Python Helpers

pythonFlatten.py: Το gaea πάει το ύψος λίμνης σε λάθος σημεία. Γίνεται flatten με python ώστε να μην υπάρχουν καρούμπαλα και το reflection να είναι όσο πιο ρεαλιστικό γίνεται.

heightmap_split.py: Εξαγωγή δεδομένων heightmap, μάσκα δάσους, μάσκα λίμνης. Σε αυτό το python αρχείο γίνεται χρήση του mesh Μέγιστης ανάλυσης ώστε να έχουμε ακριβές height και μάσκα. Αν χρησιμοποιηθεί το LOD2,3 προκύπτουν λάθη.

Seamless assets

Για να μην φαίνεται άσχημη η επανάληψη των εικόνων χρησιμοποιήθηκε αυτός ο μετατροπέας <https://www.imgonline.com.ua/eng/make-seamless-texture.php>

Βιβλιογραφία

<https://www.opengl-tutorial.org/intermediate-tutorials/billboards-particles/billboards/>

<https://3dworldgen.blogspot.com/2015/08/volumetric-clouds.html>

https://en.wikipedia.org/wiki/Viewing_frustum#:~:text=The%20region%20of%20space%20in%20the%20modeled,view%20of%20a%20perspective%20virtual%20camera%20system.

<https://learnopengl.com/Getting-started/Textures>

https://www.youtube.com/watch?v=TOPvFvL_GRY&list=WL&index=1&t

<https://www.youtube.com/watch?v=oQksg57qsRA>

<https://youtu.be/PH9q0HNBjT4>

<https://quadspinner.com/>

Assets:

Ουρανός από polyhaven + editing

Sketchfab

Cgtrader

Freepik

Stock photos